

Vector databases: A technical deep-dive



JP Hwang
Technical curriculum developer



Vector DB Comparison

by Superlinked | Last Updated : Today

Search

 Get insights

 Give us a star



Vendor ↓		About				Search				
		OSS	License	Dev Lang	VSS Launch	Filters	Hybrid Search	Facets	Geo Search	
	Weaviate	✓	BSD	go	2019	✓	✓ ⓘ	✓	✓	
	Vespa	✓	Apache-2.0	java c++	2017	✓	✓	✓	✓	
	Vectara	✗	Proprietary	-	2021	✓	✓ ⓘ	-	-	
	Vald	✓	Apache-2.0	go	2021	-	-	-	-	
	USearch	✓	Apa...	c++	2023	ⓘ	- ⓘ	✗	✓	
	Typesense	✓	GPL-3.0	c++	2023	✓	✓	✓	✓	
	txtai	✓	Apache-2.0	python	2020	✓	✓	-	-	
	Turbopuffer	✗	Proprietary	rust	2023	✓	✗	✗	✗	
	Rockset	✗	Pro...	c++	2023	✓	✓	✓	✓	
	Redis Sea...	ⓘ	(i) Redis So...	c	2021	https://superlinked.com/vector-db-comparison				
	Qdrant	✓	Apache-2.0	rust	2021					

by Superlinked | Last Updated : Today

 Get insights

 Give us a star



<https://superlinked.com/vector-db-comparison>

Vector DB Comparison

by Superlinked | Last Updated: Today

Search

Get Insights Give us a star



Vendor ↓	About				Search								Models			APIs				Ops		
	OSS	License	Dev Lang	VSS Launch	Filters	Hybrid Search	Facets	Geo Search	Multi-Vector	Sparse	BM25	Full-Text	Text Model	Image Model	Struct Model	RAG	RecSys	LangChain	Llamaind...	Managed	Pricing	In-pr...
09 Weaviate	✓	BSD	go	2019	✓	✓	✓	✓	✓	✗	✓	✓	✓	✓	-	✓	-	✓	✓	✓	-	✓
09 Vespa	✓	Apache-2.0	java c++	2017	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	https://clo...	✗
09 Vectara	✗	Proprietary	-	2021	✓	✓	-	-	-	✗	✗	✗	✗	-	-	✓	-	✓	✓	✓	Pricing - V...	✗
09 Vald	✓	Apache-2.0	go	2021	-	-	-	-	-	✗	-	✗	-	-	-	-	-	✓	✗	✗	-	✗
09 USearch	✓	✗ Apa...	c++	2023	✓	✗	✗	✓	✓	-	✗	✓	✓	✓	✓	-	-	✓	✓	✗	-	✓
09 Typesense	✓	GPL-3.0	c++	2023	✓	✓	✓	✓	✓	-	✗	✓	✓	✓	✓	-	-	✓	-	✓	Pricing T...	✗
09 txtai	✓	Apache-2.0	python	2020	✓	✓	-	-	✓	✗	✓	-	✓	✓	-	✓	✓	-	-	-	-	✓
09 Turbopuffer	✗	Proprietary	rust	2023	✓	✗	✗	✗	✗	✗	✗	✗	✗	✗	-	-	✗	✗	✗	✓	\$1/million ...	✗
09 Rocksdb	✗	✗ Pro...	c++	2023	✓	✓	✓	✓	✗	✗	✗	✓	✓	✓	✗	✗	✗	✓	✓	✓	-	✗
09 Redis Sea...	✓	(i) Redis So...	c	2021	✓	-	✓	✓	✓	✗	✓	✓	✗	-	-	-	-	✓	✓	✓	-	✗
09 Qdrant	✓	Apache-2.0	rust	2021	✓	✗	✗	✓	✓	✓	✗	✓	✓	✗	-	-	✓	✓	✓	✓	Qdrant Pri...	✓
09 Pinecone	✗	Proprietary	rust	2019	✓	✓	✗	✗	✗	✓	✗	✗	✓	✗	-	-	✓	✓	✓	✓	-	✗
09 pgvector	✓	✗ Post...	c	2021	✓	✓	-	✓	✓	✗	✓	✓	✗	-	-	-	✓	✓	✓	✓	-	✗
09 pgvector.rs	✓	✗ Apa...	rust	2023	✓	✓	-	✓	✓	✓	✓	✓	-	-	-	-	-	✓	✓	-	-	✗
09 OramaSe...	✓	Apache-2.0	typescript	2022	✓	✗	✓	✓	✗	✗	✓	✓	✓	-	-	-	-	✗	✗	✓	free plan / ...	✓
09 OpenSear...	✓	Apache-2.0	java	2021	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	-	✓	✓	✓	-	✗
09 Nucleia DB	✓	AGPLv3	rust python	2021	✓	✓	✓	✗	-	✗	✓	✓	-	-	-	✓	✗	✓	✗	✓	-	✗
09 Neo4j	✗	✗ Pro...	java scala	2023	✓	✓	✗	✓	✓	✗	✗	✓	✗	✗	✓	✓	✓	✓	✓	✓	-	✓
09 MyScale	✓	Apache-2.0	c++	2023	✓	✓	✓	✓	✓	✗	✓	✓	✓	✗	✗	✗	✗	✓	✓	✓	MyScale P...	✗
09 MongoDB...	✗	✗ SSP...	c++ java	2023	✓	✓	✓	✓	✓	✗	✓	✓	✗	-	-	-	-	✓	✓	✓	MongoDB ...	✗
09 Milvus	✓	Apache-2.0	go c++	2019	✓	✓	-	✗	✓	✓	✓	✓	✓	-	-	-	-	✓	✓	✓	Zilliz Cloud	✗
09 Meilisearch	✓	✗ MIT	rust	2023	✓	✓	✓	✓	✓	✗	✓	✓	✓	✓	-	-	-	✓	✗	✓	Meilisearc...	✗
09 Marqo	✓	Apache-2.0	python	2022	✓	✓	-	-	✓	✗	✓	✗	✓	✓	-	-	-	✓	✗	✓	Pricing - ...	✗
09 LanceDB	✓	Apache-2.0	rust	2023	✓	✓	-	-	✗	-	✓	✓	✓	-	-	-	-	✓	✓	✓	-	✓
09 KDB.AI	✗	Proprietary	python	2023	✓	✓	✓	-	-	✓	✓	-	-	-	-	-	-	✓	✓	✓	Cloud = Free: 4 G...	✗
09 GCP Vert...	✗	-	-	2021	✓	✗	-	-	✗	✗	✗	✗	✗	-	-	-	-	✓	✗	✓	-	✗
09 Epsilla	✓	GPL-3.0	c++	2023	✓	✓	-	-	✓	✓	-	-	✓	-	-	✓	✓	✓	✓	✓	Storage: \$...	✗
09 Elasticsearch...	✗	Elastic Lice...	java	2021	✓	✓	✓	✓	✓	✓	✓	✓	✓	-	-	-	-	✓	✓	✓	-	✗
09 DataStax ...	✓	Proprietary	java go	2023	✓	✓	✓	✓	✗	✗	✗	✓	✓	✗	-	✓	✓	✓	✓	✓	https://ww...	✗
09 CrateDB	✓	Apache 2.0	java	2023	✓	-	✓	✓	✓	-	✓	✓	-	-	-	-	-	✓	-	✓	CrateDB P...	-
09 ClickHouse	✓	Apache 2.0	c++	2022	✓	✗	✓	✓	✓	✗	✗	✓	✓	✓	✓	✗	✗	✓	✓	✓	Clickhous...	✓
09 Chroma	✓	Apache-2.0	python	2022	✓	✗	-	-	✗	✗	✗	✗	-	-	-	-	-	✓	✓	✗	-	✗
09 Azure AI S...	✗	Proprietary	c++	2023	✓	✓	✓	✓	✓	✗	✓	✓	✓	✓	-	✓	✗	✓	✓	✓	Pricing - A...	✗
09 ApertureDB	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	✗
09 Apache S...	✓	Apache-2.0	java	2022	✓	✓	✓	✓	✓	-	✓	✓	-	-	-	-	-	-	-	-	-	✗
09 Apache C...	✓	Apache-2.0	java	2023	✓	✓	-	-	-	✗	-	✗	-	-	-	-	-	-	-	-	-	✗
09 Anari AI	✗	Proprietary	-	2023	-	✗	-	✗	-	-	✗	-	-	-	-	-	-	-	-	-	-	✗
09 Activaloo...	✓	MPL 2.0	python c++	2023	✓	✓	-	✗	✓	✓	✗	✓	✓	✓	-	-	-	✓	✓	✓	Free 200G...	✓

https://superlinked.com/vector-db-comparis

<https://superlinked.com/vector-db-comparison>



Vector databases are hot!

But why?



Login

Search 🔍

Startups

Venture

Security

AI

Crypto

Apps

Events

Startup Battlefield

More

AI

Why vector databases are having a moment as the AI hype cycle peaks

GenAI spurs demand for vector search startups, but database giants are also taking note

Paul Sawers / 8:00 AM PDT • April 20, 2024

 Comment

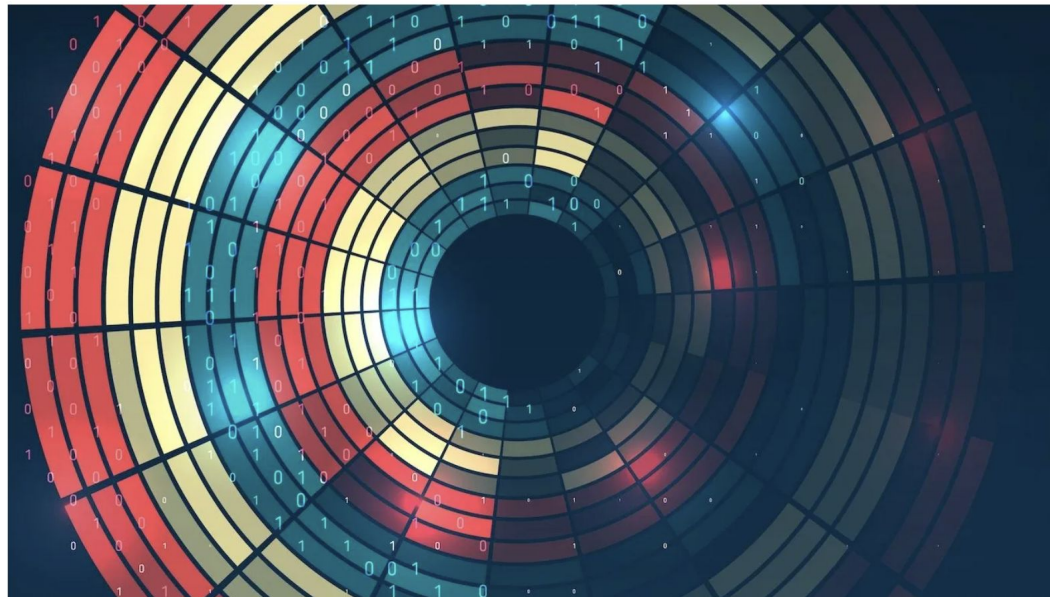


 Image Credits: Getty Images





Vector databases are hot!

But **why** does AI need a new type of DB?

What makes vector databases **different**?

Will it be **useful for me**?



Let's find out by...



Building **our own solution**

(At least **conceptually!**)

Conceptually build a vector-store

- Start small
- Add components
 - Learn their jobs
- Why do this?
 - Knowledge!
 - Tool selection and usage



Vectors:

A refresher / An introduction



A vector is

A vector is a set of numbers

Like

`[1, 0]`

or

`[0.513, 0.155, 0.983, 0.001, 0.932]`

or

`[0.0009420722, 0.020158706, -0.03939992,
-0.025480185, 0.018441677, 0.0023035712,
-0.012281344, -0.025270471, -0.056622636, ...]`

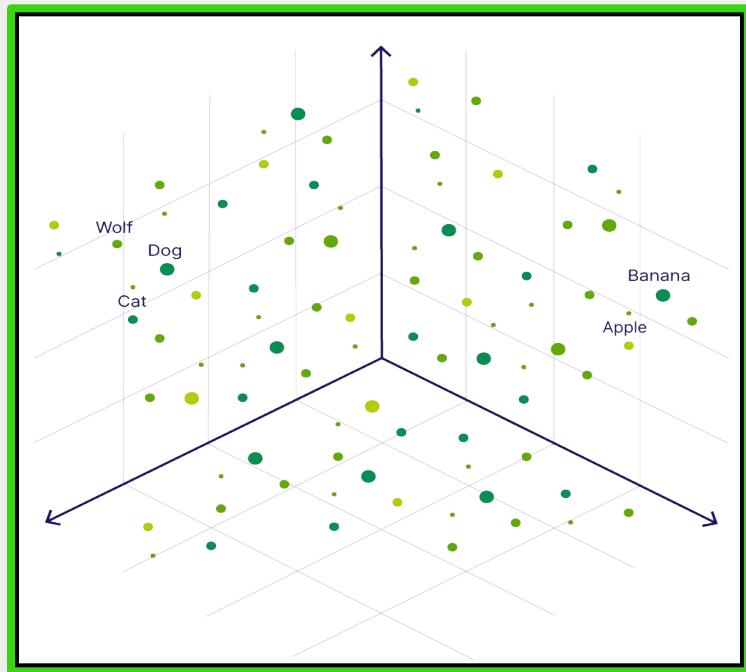
Why does this matter? 🤔



Vectors can **represent meaning**.

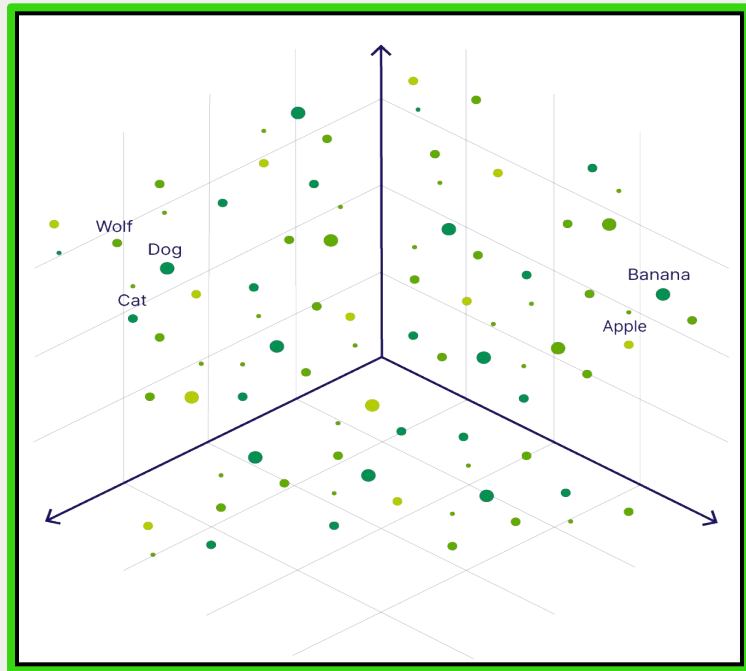
Why vectors?

- Vectors close if meaning close
- **Vector search is similarity search**



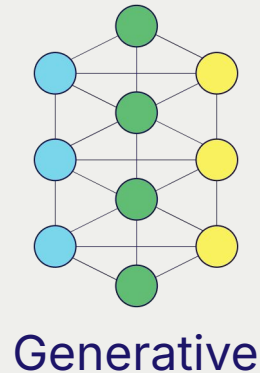
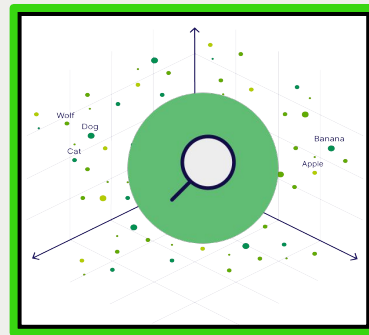
Why vectors?

- Vectors close if meaning close
- Vector search is similarity search
- **Great for finding contextually relevant info**



Why vectors?

- Vectors close if meaning close
- Vector search is similarity search
- Great for finding contextually relevant info
 - **For working with GenAI**

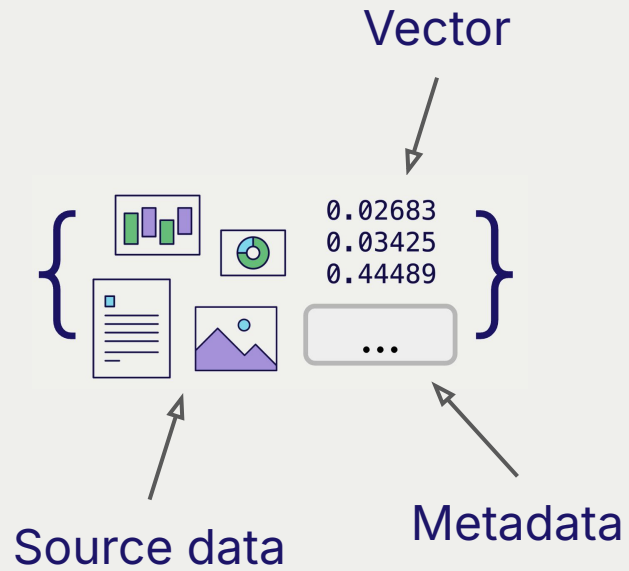


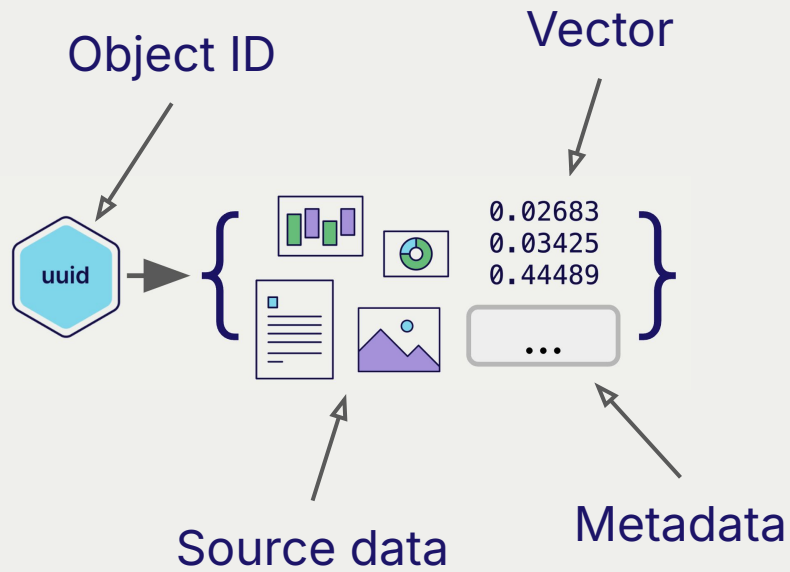


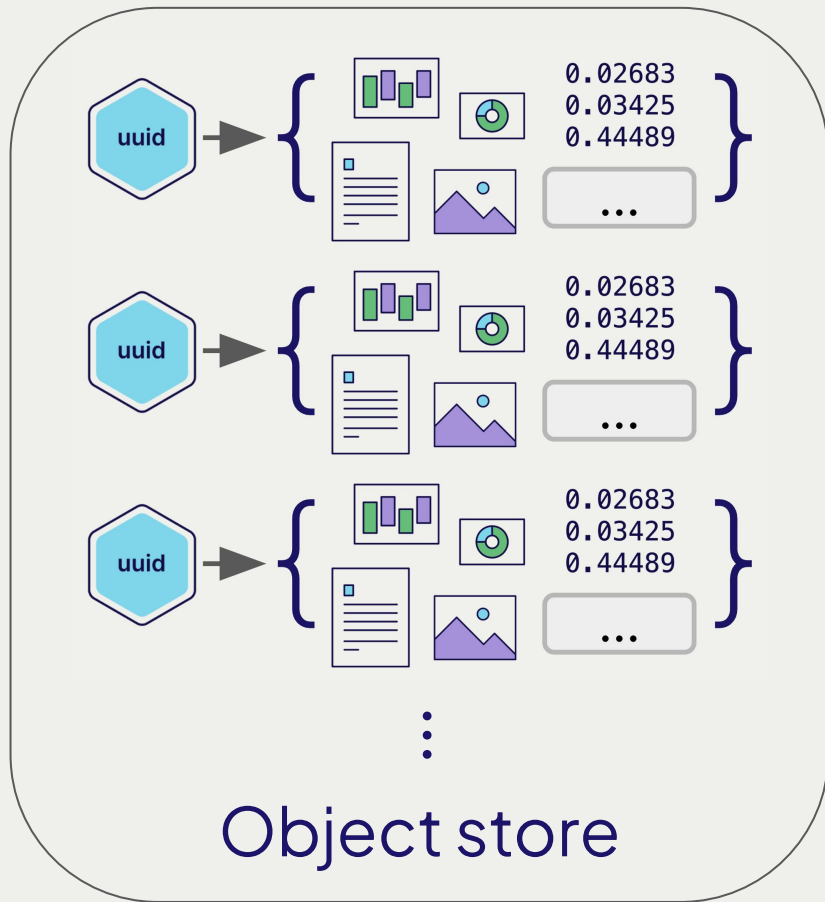
Let's build **a vector store**



Source data



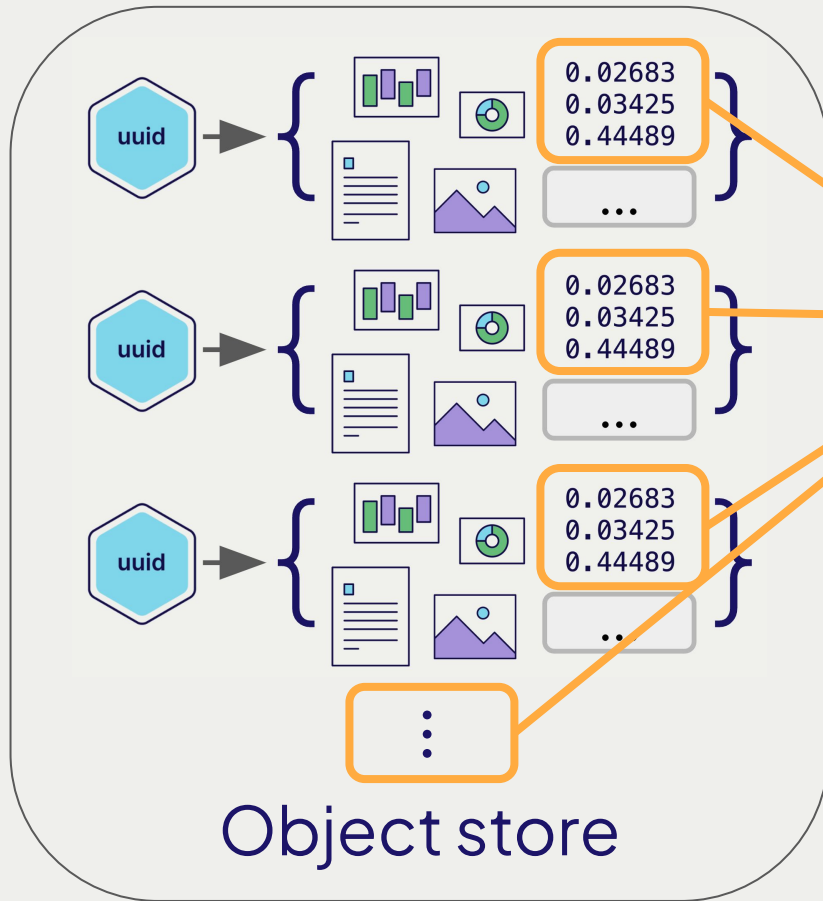




User perspective:

- Can it save desired:
 - Data types
 - Vectors
 - Metadata

(Like many other key/value stores)

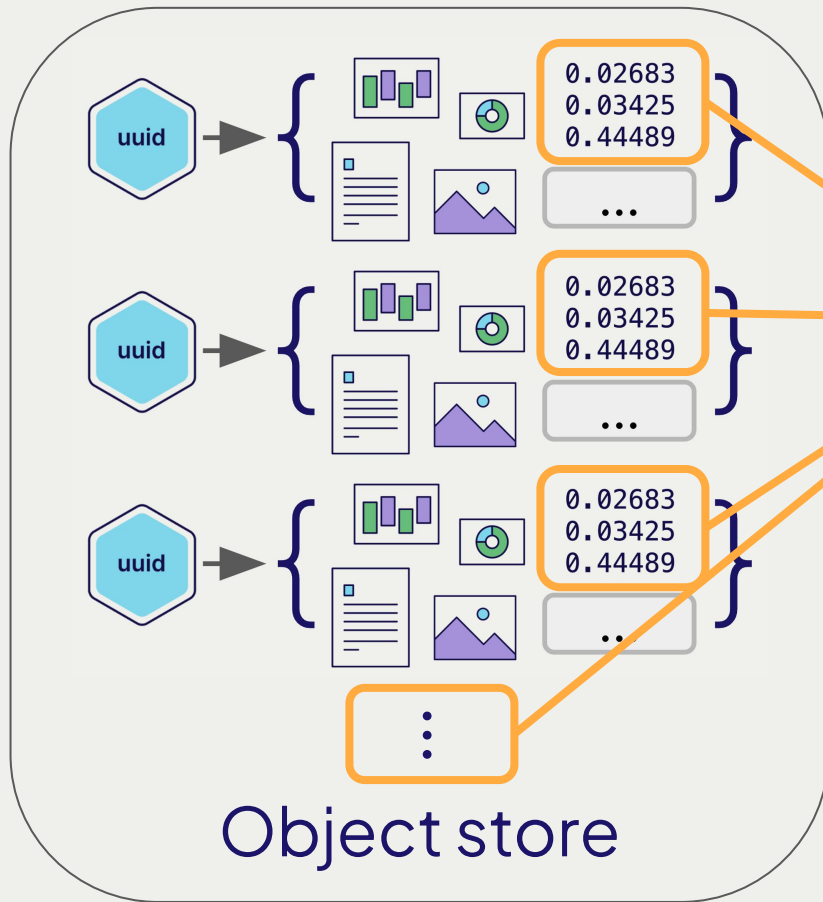


Solution:

- Calculate vector similarity
vs query

Limitation:

- Retrieve the right
information



Solution:

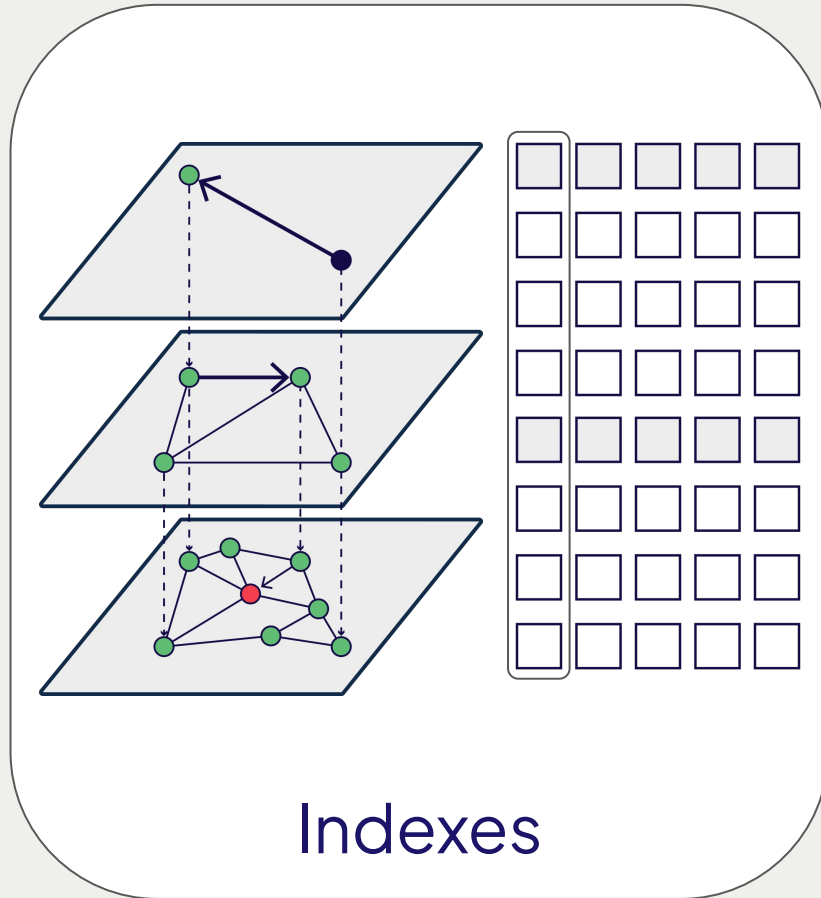
- Calculate vector similarity
- vs query
- **Expensive!**

Limitation:

- Retrieve the right information

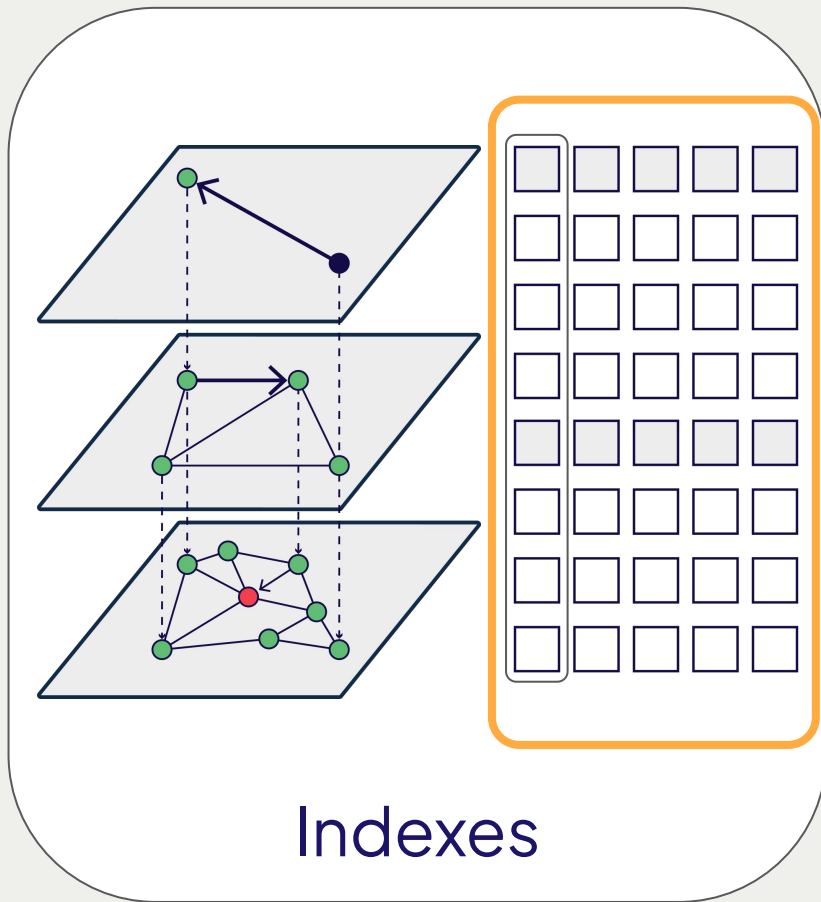


How do we **scale search?**



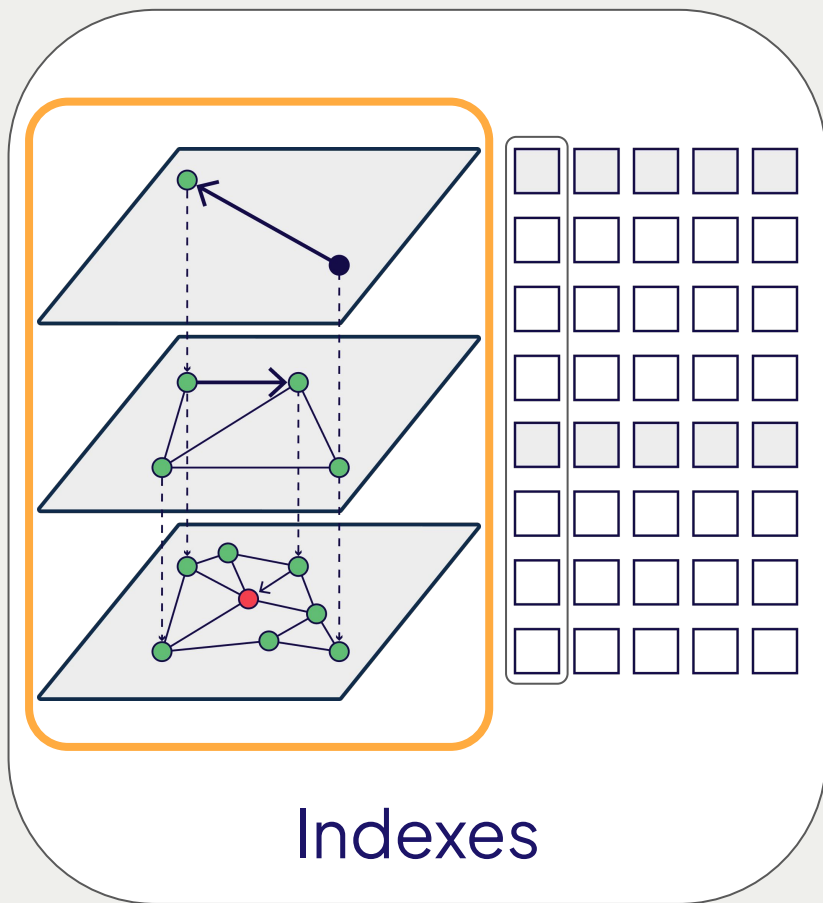
Indexes:

- Catalogue of data
 - Can speed up search
- Exist in RDBMS



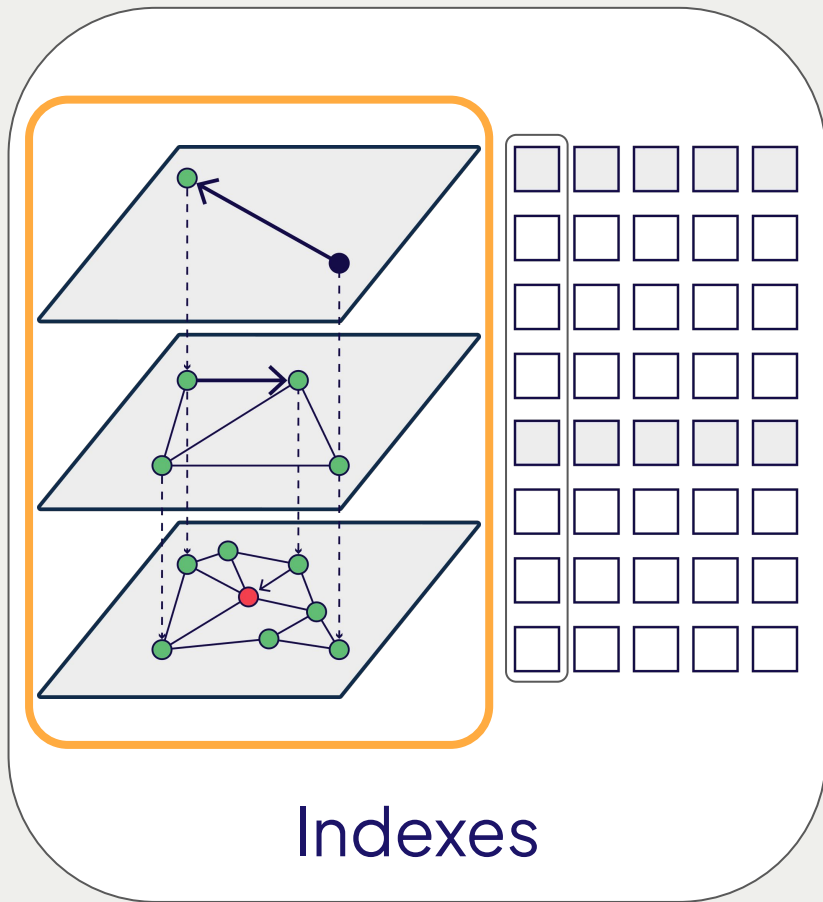
Inverted indexes:

- Organized information, by:
 - Property
 - Metadata (e.g. timestamp)



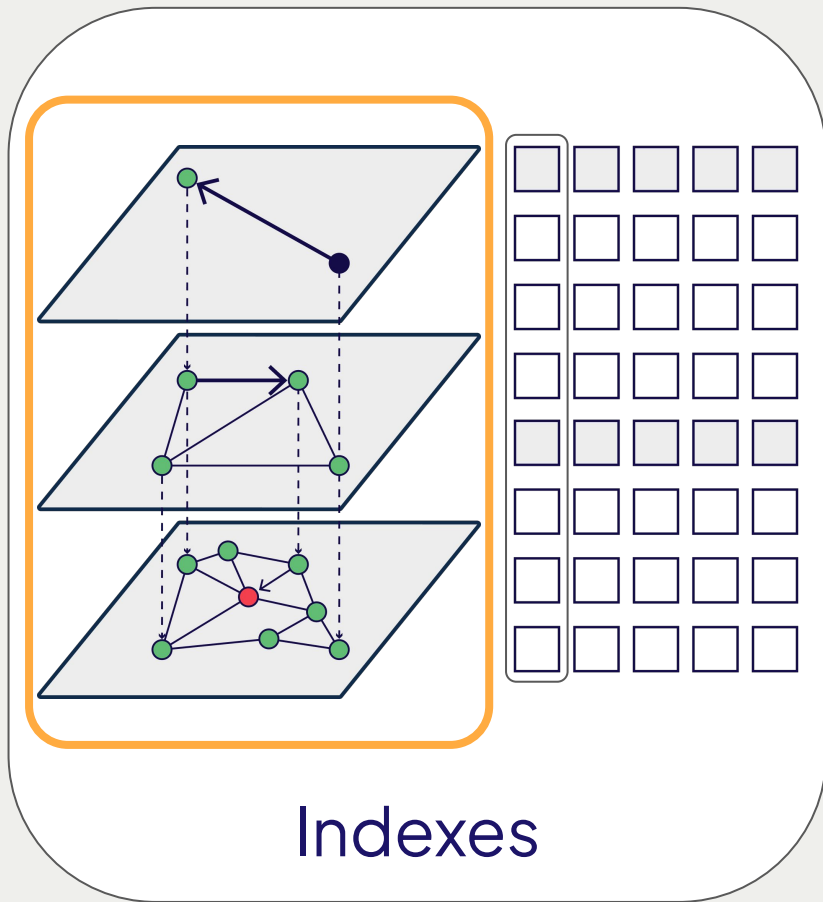
Vector indexes:

- Vectors capture meaning



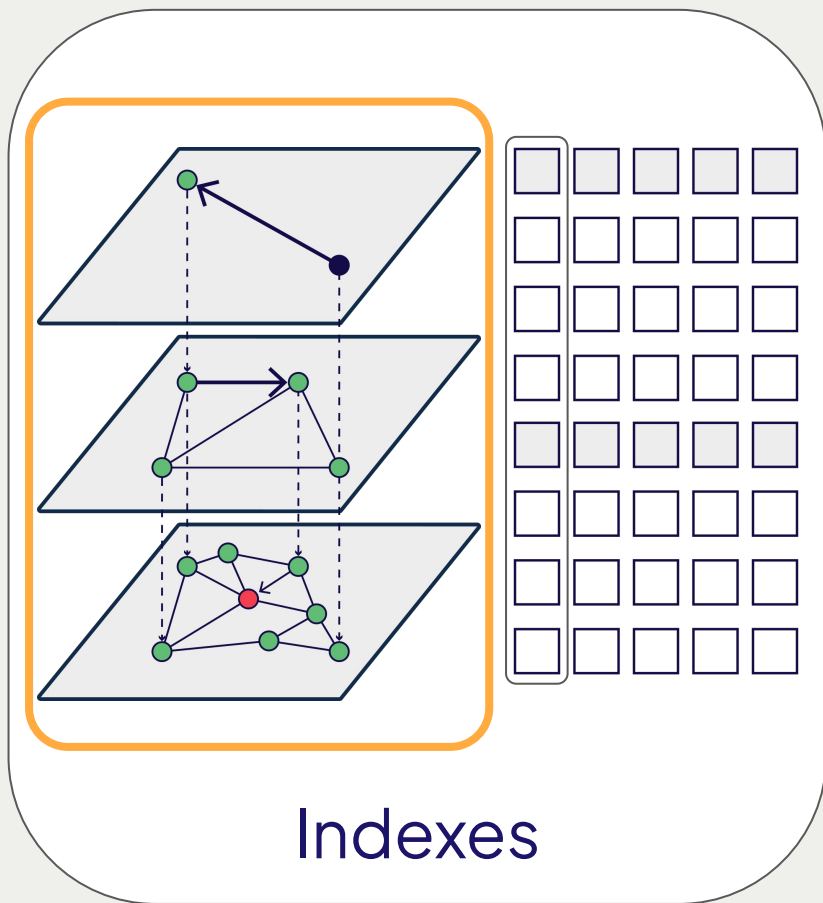
Vector indexes:

- Must be organised by *similarity*



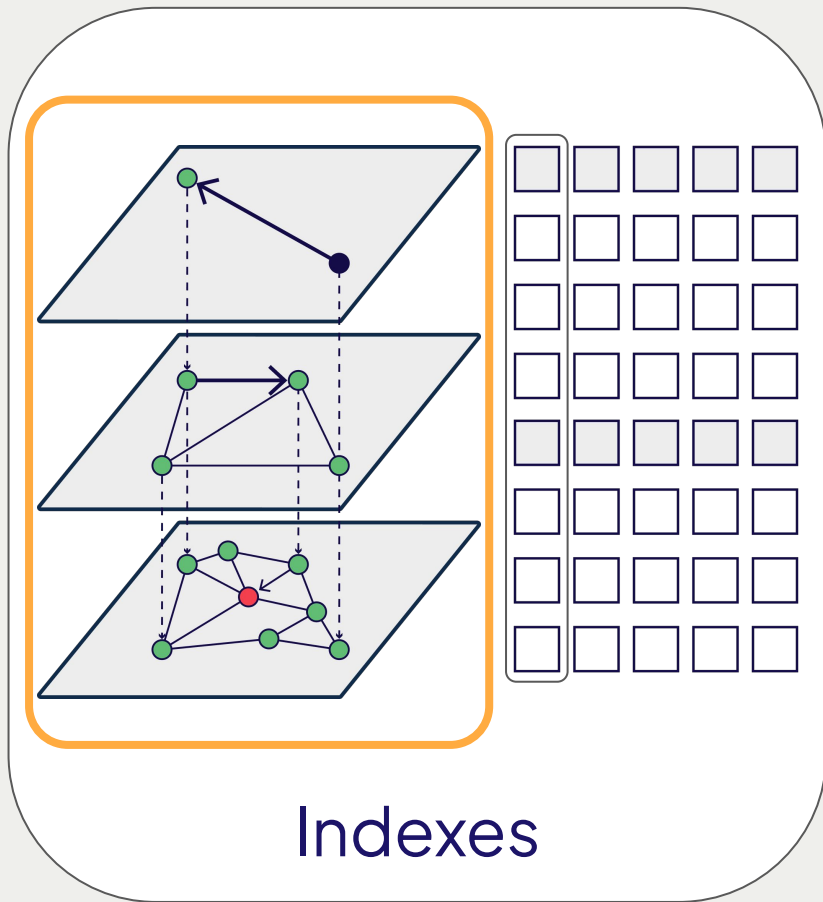
Vector indexes:

- Must be organised by *similarity*
- Types:
 - Flat
 - Graph-based (HNSW)
 - Tree-based
 - Others



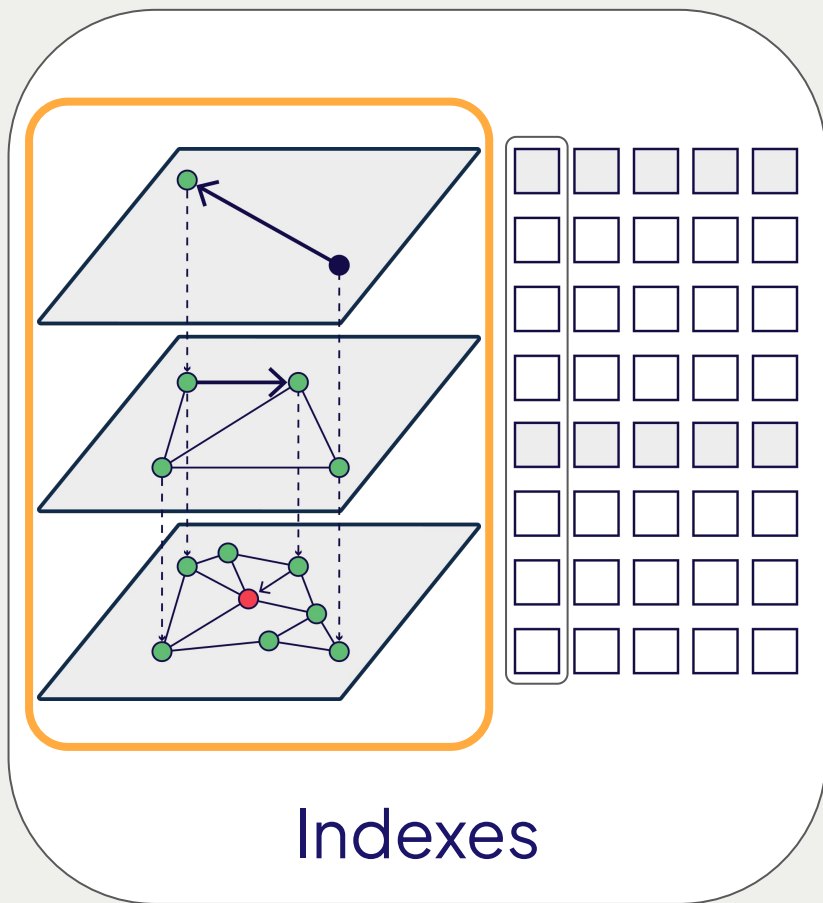
Vector indexes:

- "Approximate" searches
- Trade-offs
 - Speed
 - Accuracy
 - Memory



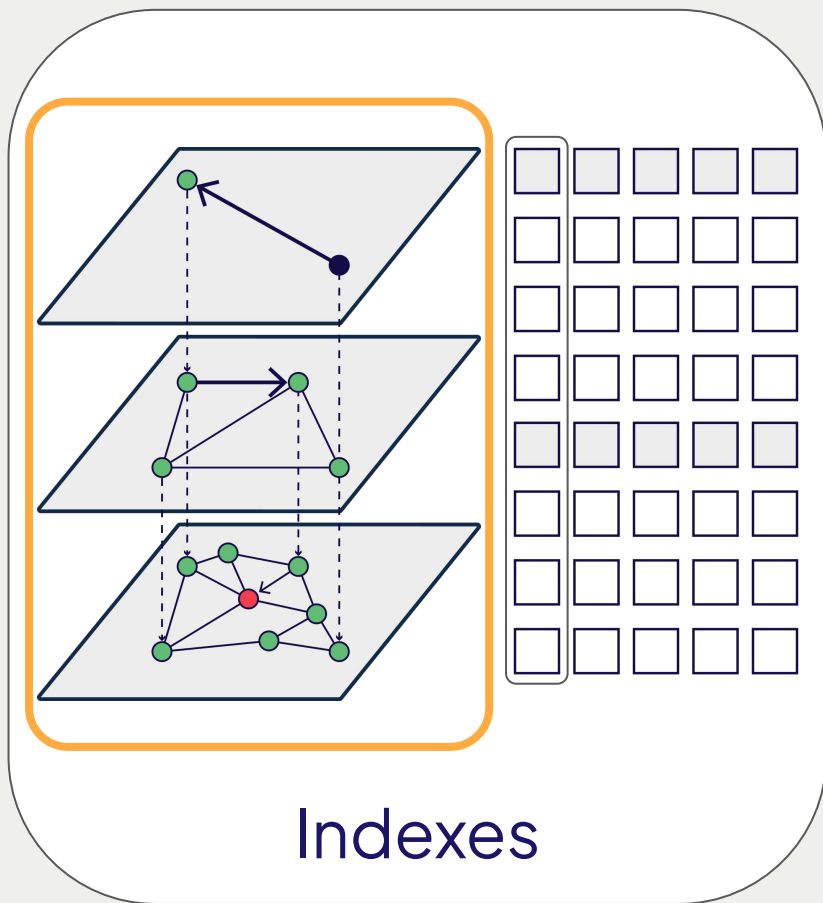
Flat indexes:

- Low overhead
- Linear search
- Good for small sets
- Does not scale well - $O(n)$



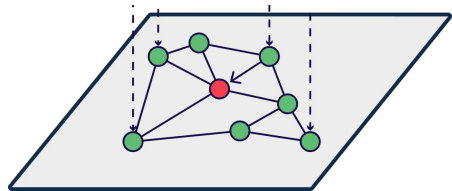
Graph-based indexes:

- Common
 - HNSW most common
- Scalable
- Typical limitation:
 - Memory footprint



Other index types:

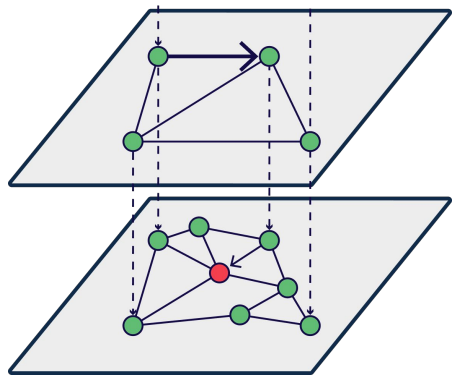
- Numerous (ANNOY, IVF)
- Have some benefits
- Generally not as scalable as HNSW
- Not as popular



HNSW

HNSW in focus

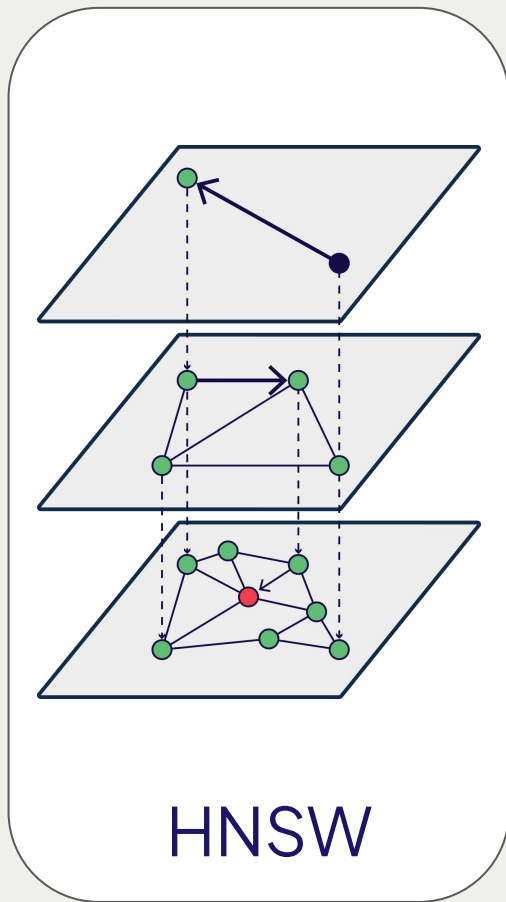
- Layered graphs
- Lowest layer: all vectors (nodes)
 - Nodes connected to neighbors



HNSW

HNSW in focus

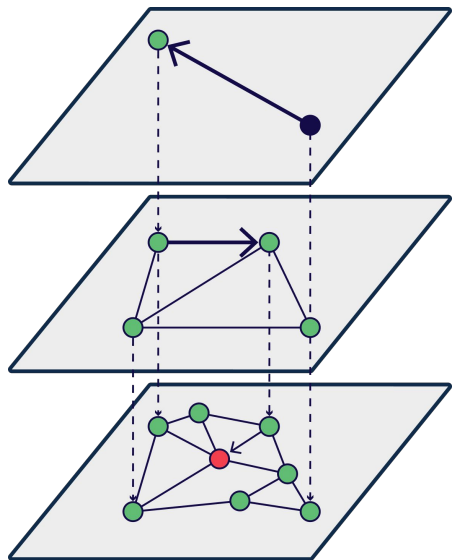
- Layered graphs
- Lowest layer: all vectors (nodes)
 - Nodes connected to neighbors
- Upper layers: subsets of nodes



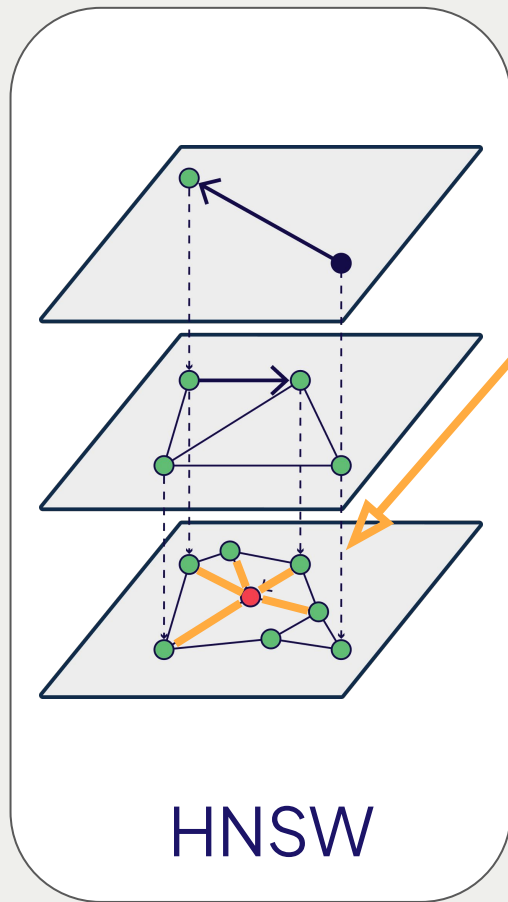
HNSW in focus

- Layered graphs
- Lowest layer: all vectors (nodes)
 - Nodes connected to neighbors
- Upper layers: subsets of nodes
 - Higher $\leftrightarrow$ fewer nodes
 - Higher $\leftrightarrow$ faster traversal
- Multi-dimensional skip lists

HNSW parameters

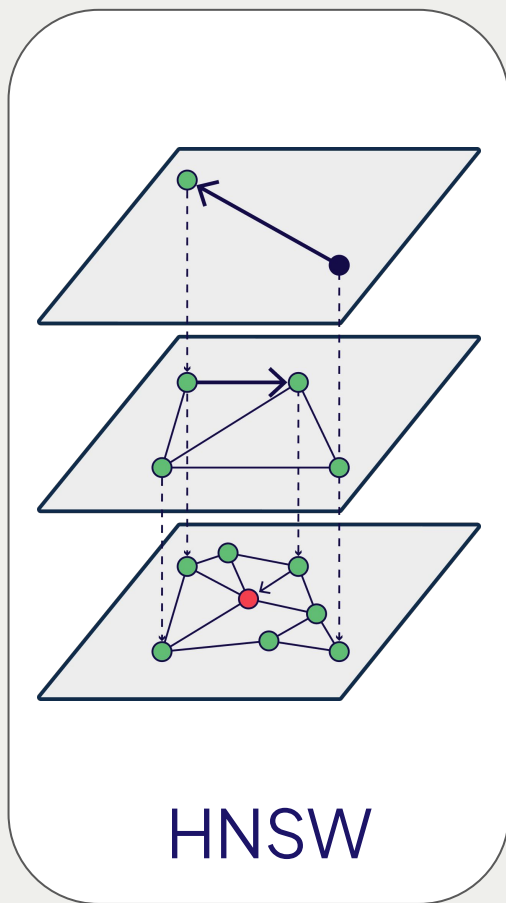


HNSW



HNSW parameters

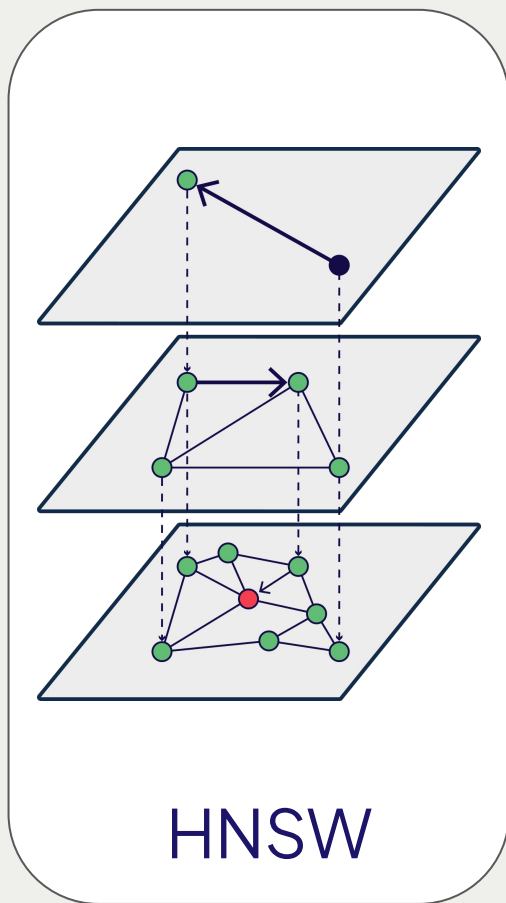
- **maxConnections (m)**
 - N (node → node) connections
 - e.g. 32
 - Note: bottom layer → $m * 2$



HNSW parameters

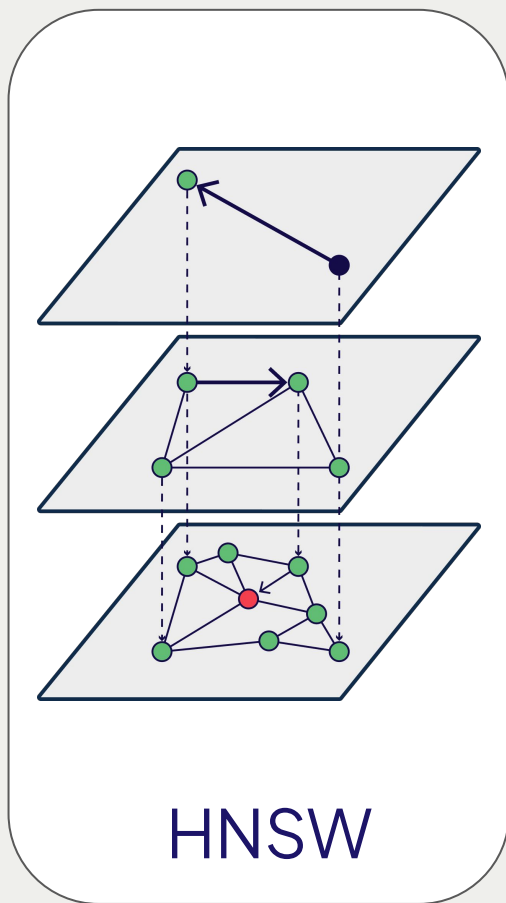
- maxConnections (m)
- **ef** → size of list during search
 - Higher → better
 - Cost: slower

rank	object	distance
1	<uuid>	0.0514
2	<uuid>	0.0642
...	...	...
...	...	...
...	...	...
...	...	...
max(ef, limit)	<uuid>	0.3532



HNSW parameters

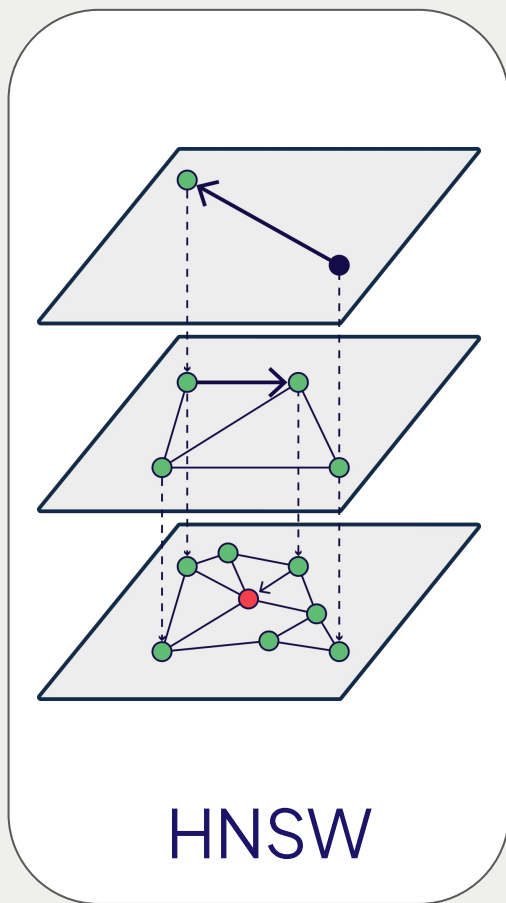
- maxConnections (m)
 - ef
 - **efConnections**
 - For graph construction
 - Higher $\rightarrow$ better
- Slower insertion



HNSW parameters

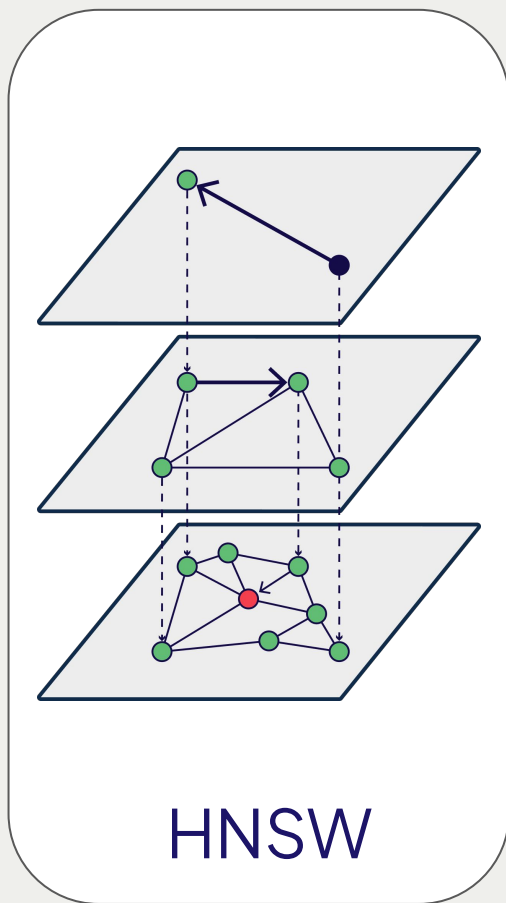
- maxConnections (m)
- ef
- efConnections

Tune speed vs recall



HNSW in focus

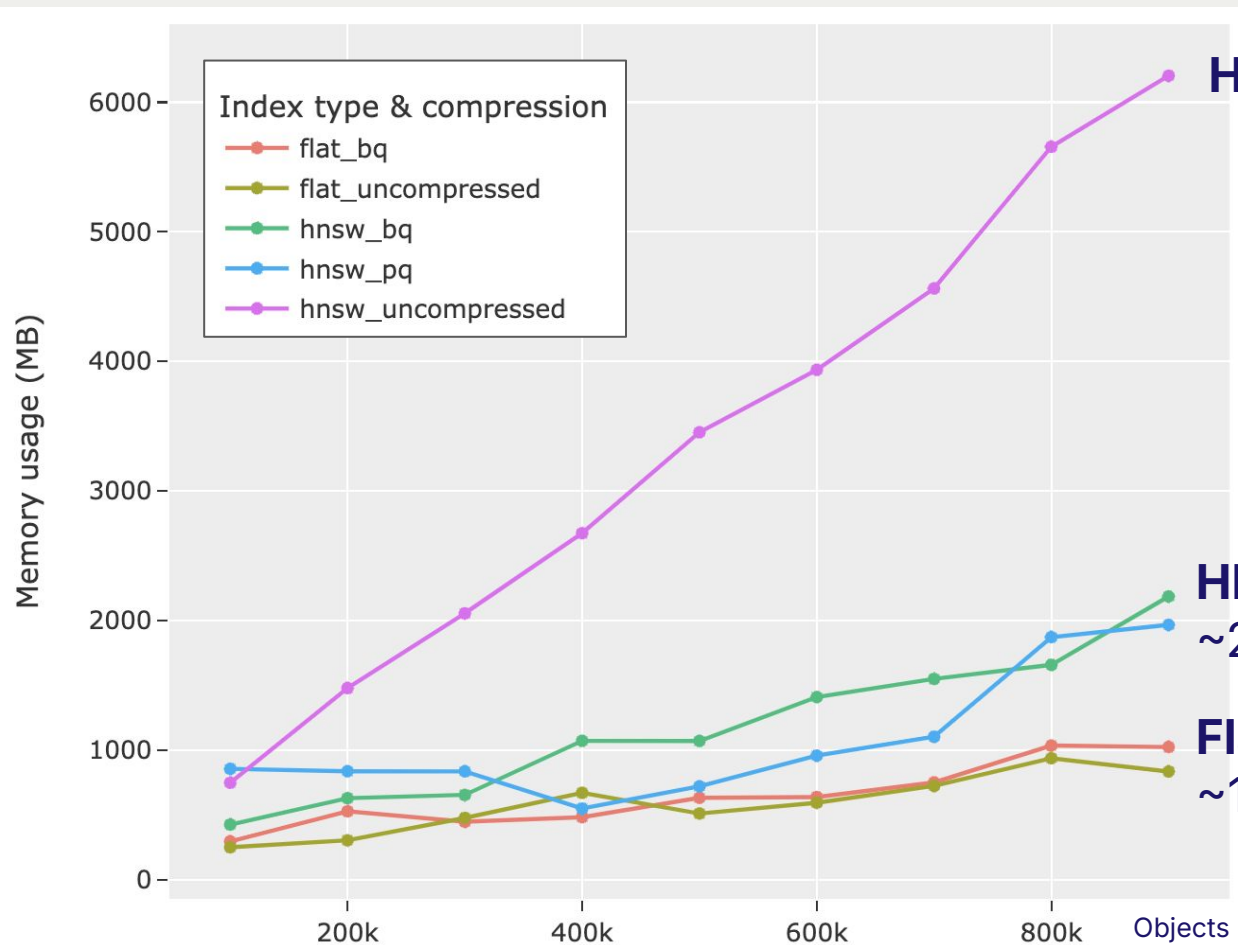
- In-memory data structure
- Typical limitation: memory footprint
 - Vectors ($\sim 4\text{B}/\text{dim}$ @ ~ 2000 dims)
 - Connections ($\sim 10\text{B}/\text{ea}$ @ 32-64)
 - @10M objects $\rightarrow \sim 65\text{GB}$
 - @100M $\rightarrow \sim 650\text{GB}$, etc...



Quantization

- Compression
- Reduce precision from floats
 - To integers, or even binary
- Reduce number of dimensions
 - Product quantization

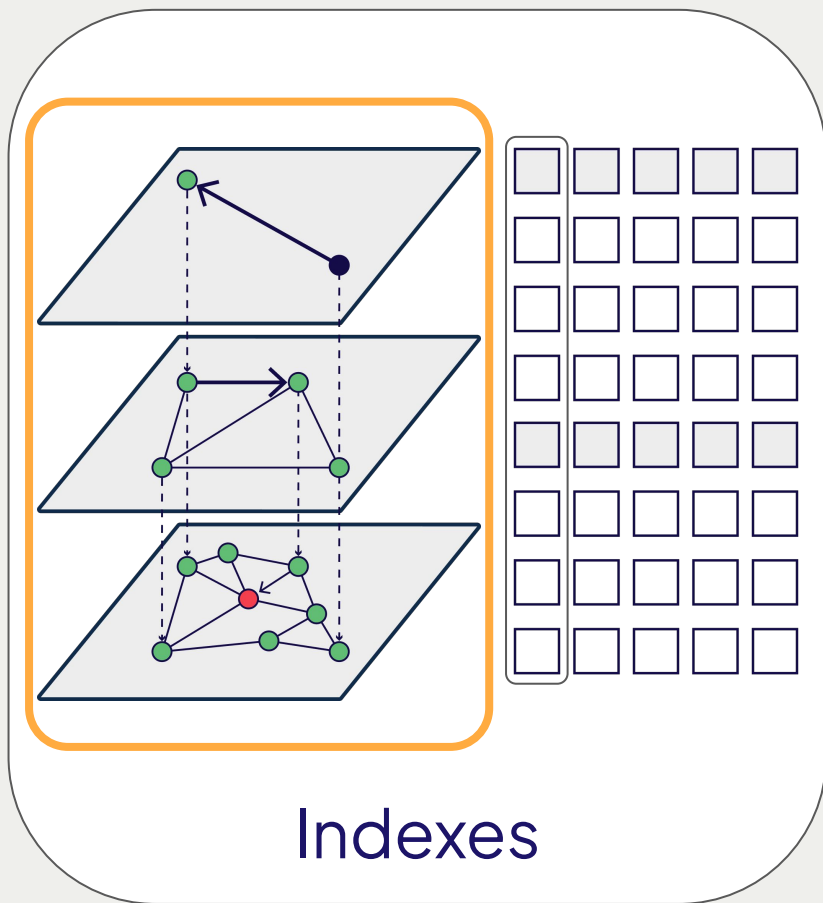
Example: Index type & quantization vs memory footprint



HNSW: ~6GB

HNSW + BQ/PQ:
~2GB

Flat (+BQ):
~1GB

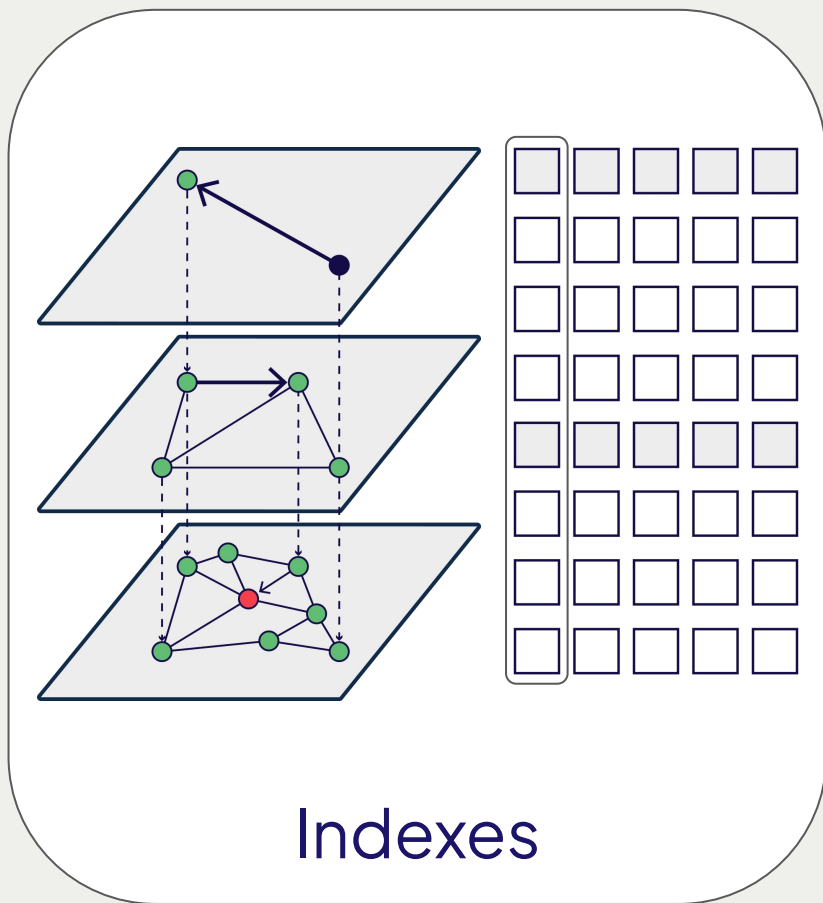


User perspective:

- Two indexes
- Many options to tune
 - Speed, recall & memory
 - Index choice
 - Index parameters
 - Quantization (precision)

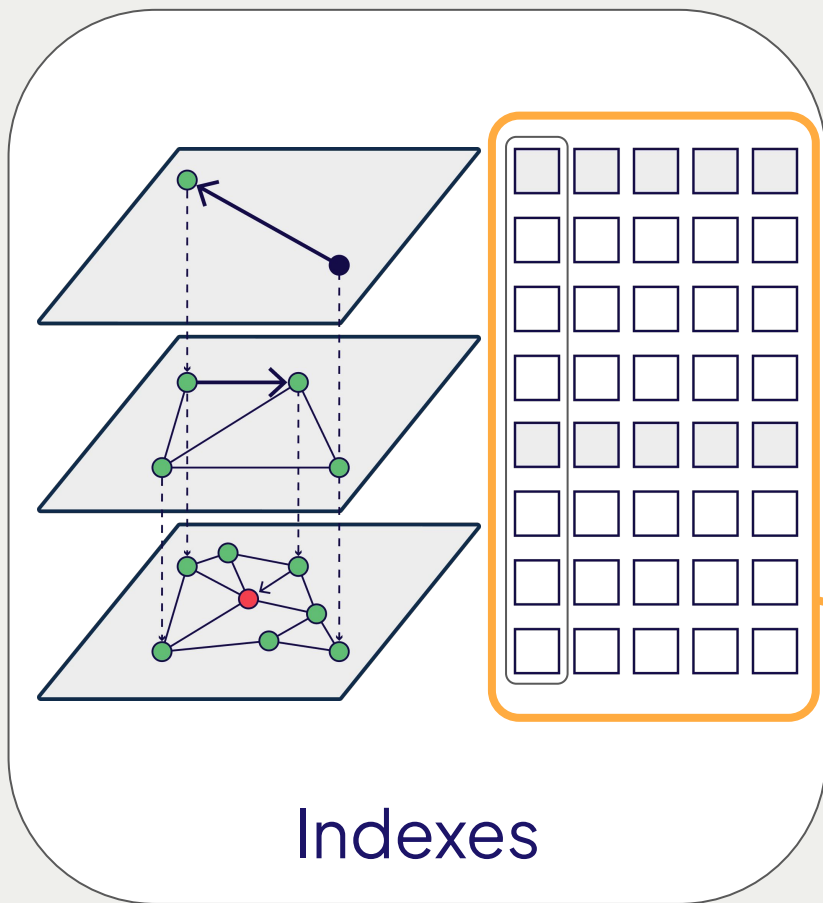


Tricks with **both indexes**



Filtering:

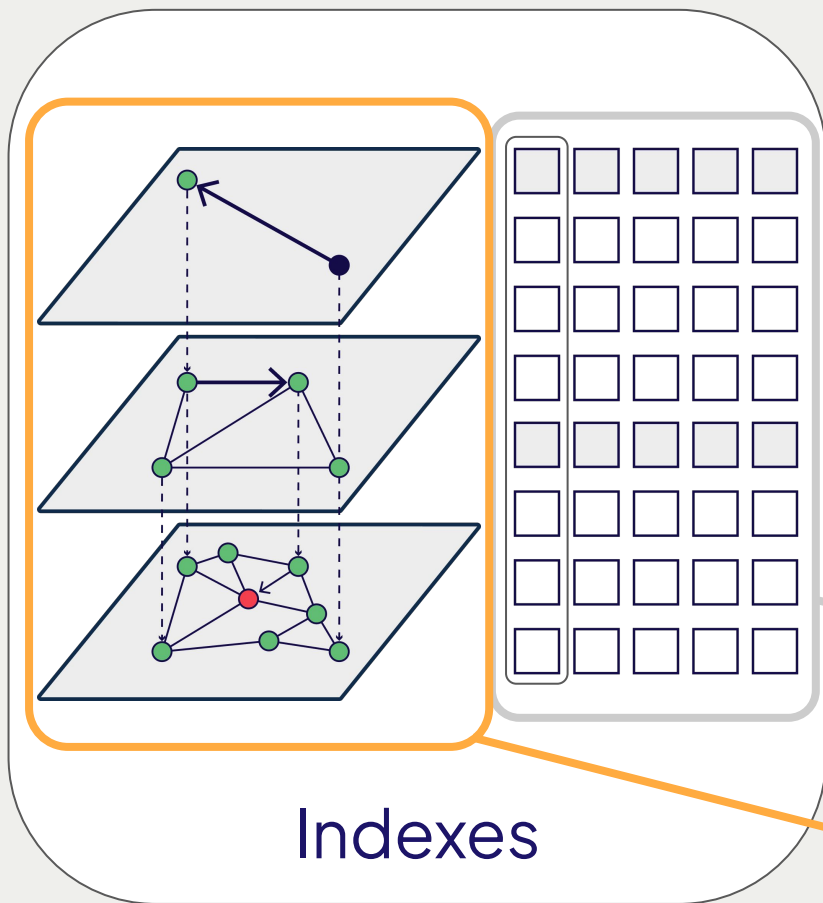
- Search with criteria, e.g.
 - where 'source' == 'Wiki'



Filtering:

- Search with criteria, e.g.
 - where 'source' == 'Wiki'

① Build "allow list"

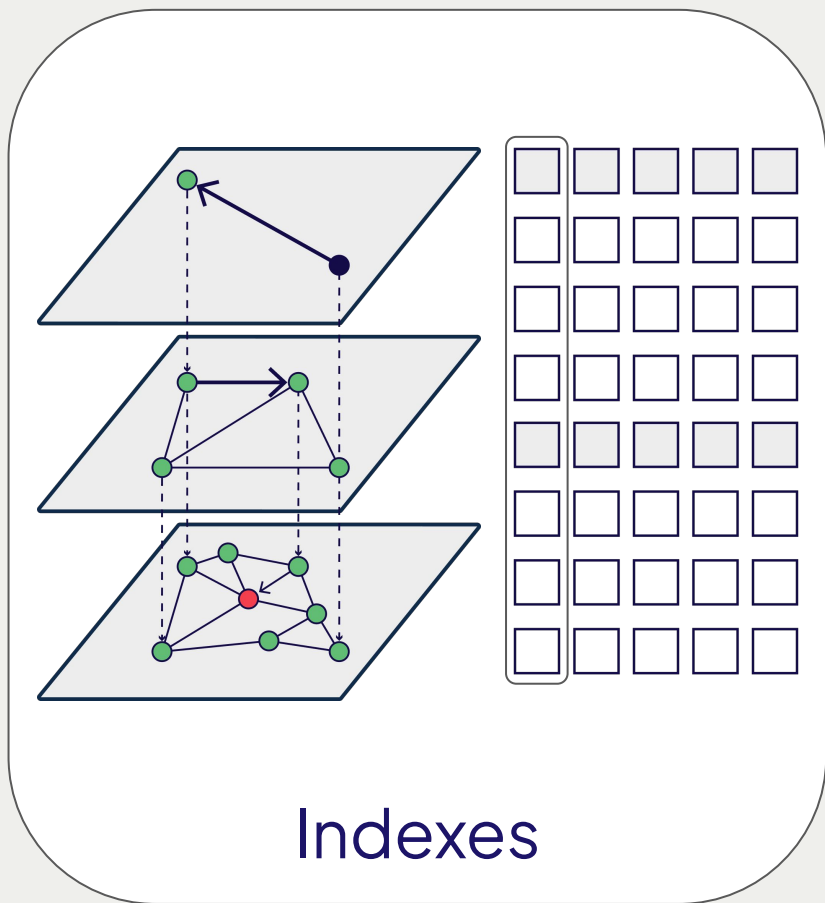


Filtering:

- Search with criteria, e.g.
 - where 'source' == 'Wiki'

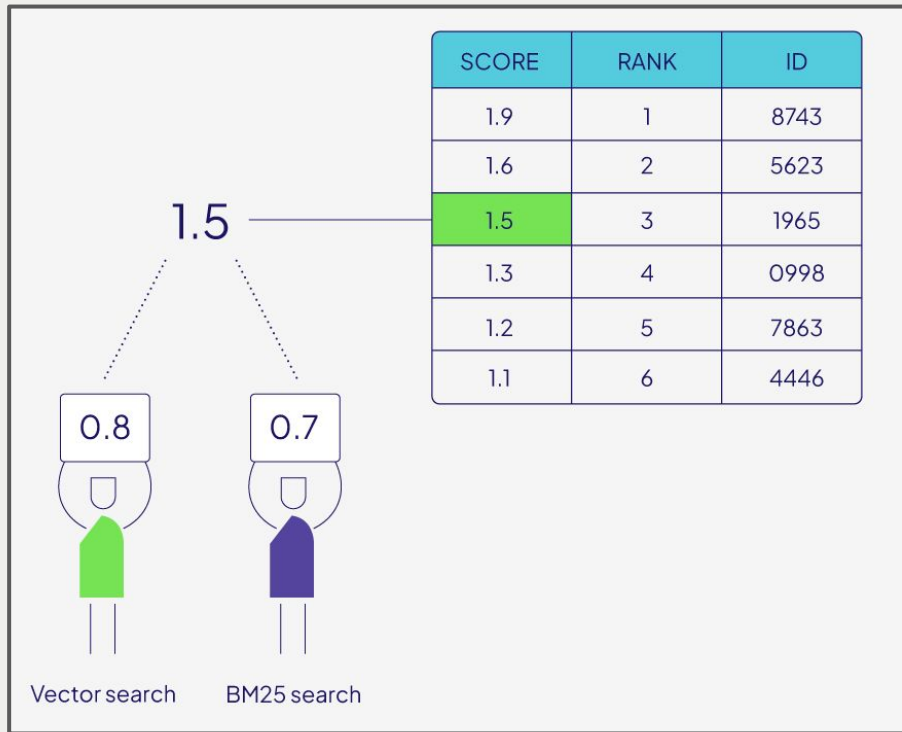
① Build "allow list"

② Vector search with
"allow list" context



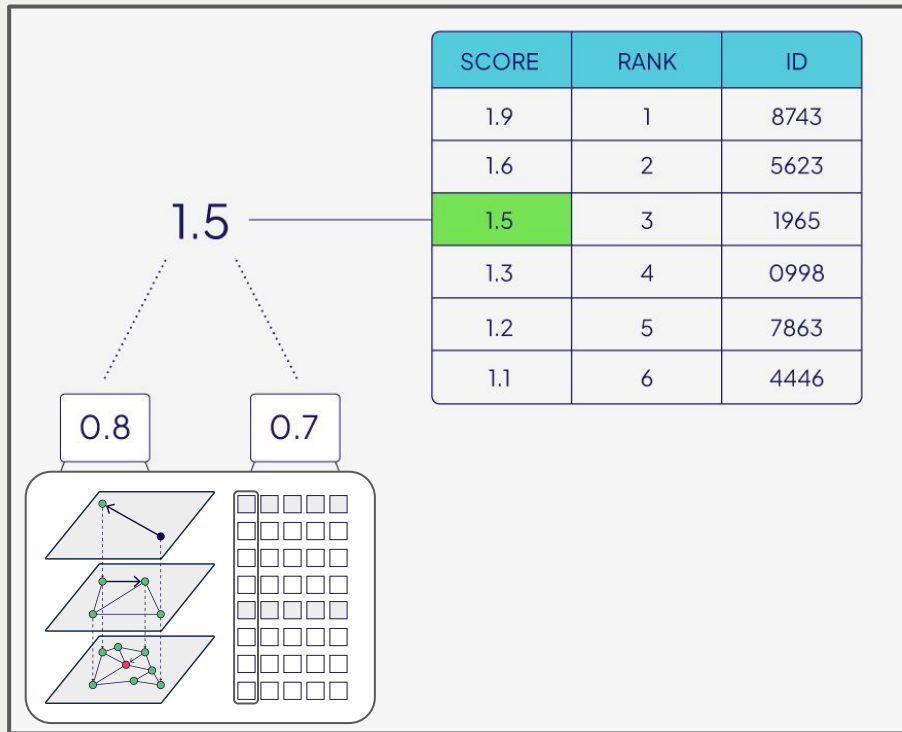
Filtering:

- Search with criteria, e.g.
 - where 'source' == 'Wiki'
- Requires speed
 - Allow list:v big/v small
 - Separate filterable index
 - Roaring bitmaps



Hybrid search:

- Combine keyword & vector search results

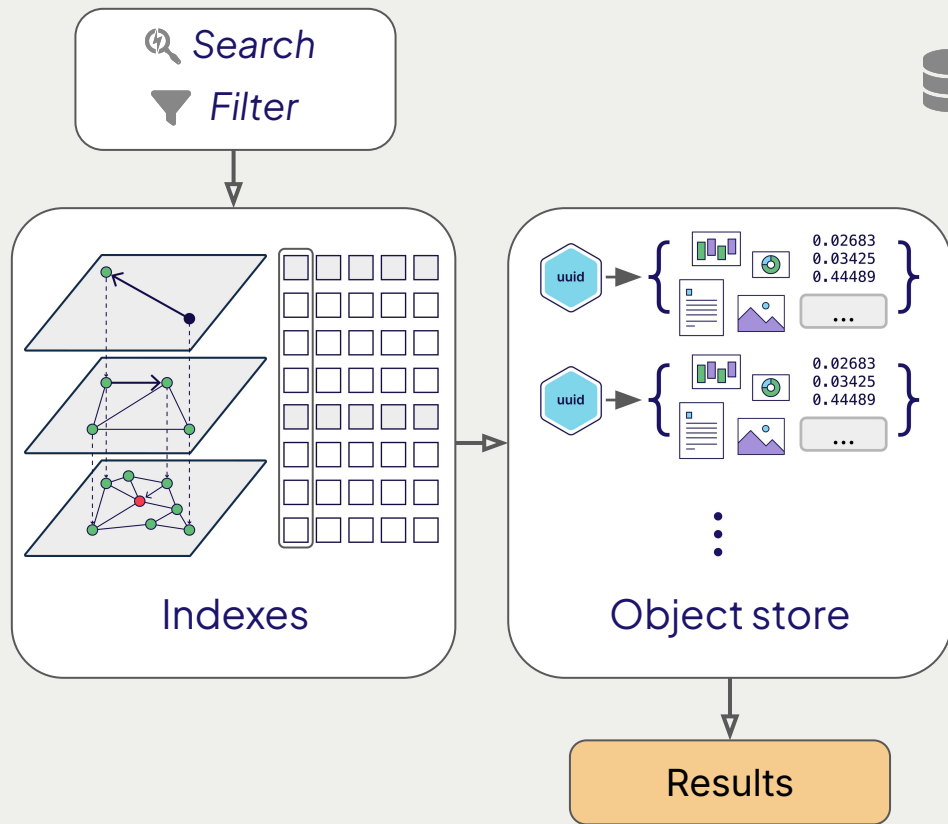


Hybrid search:

- Combine keyword & vector search results
- Each search uses a different index



Progress update / recap

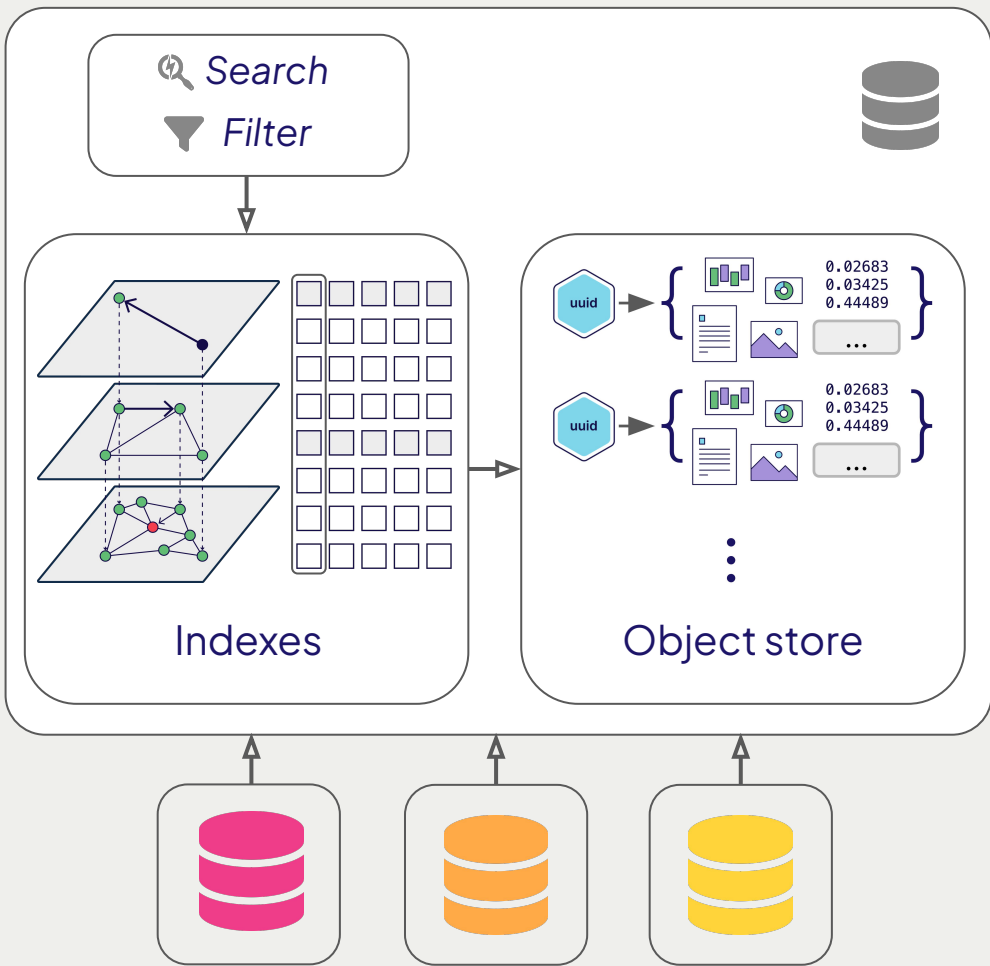


Mini-database:

- Store data
- Fast searches & filters

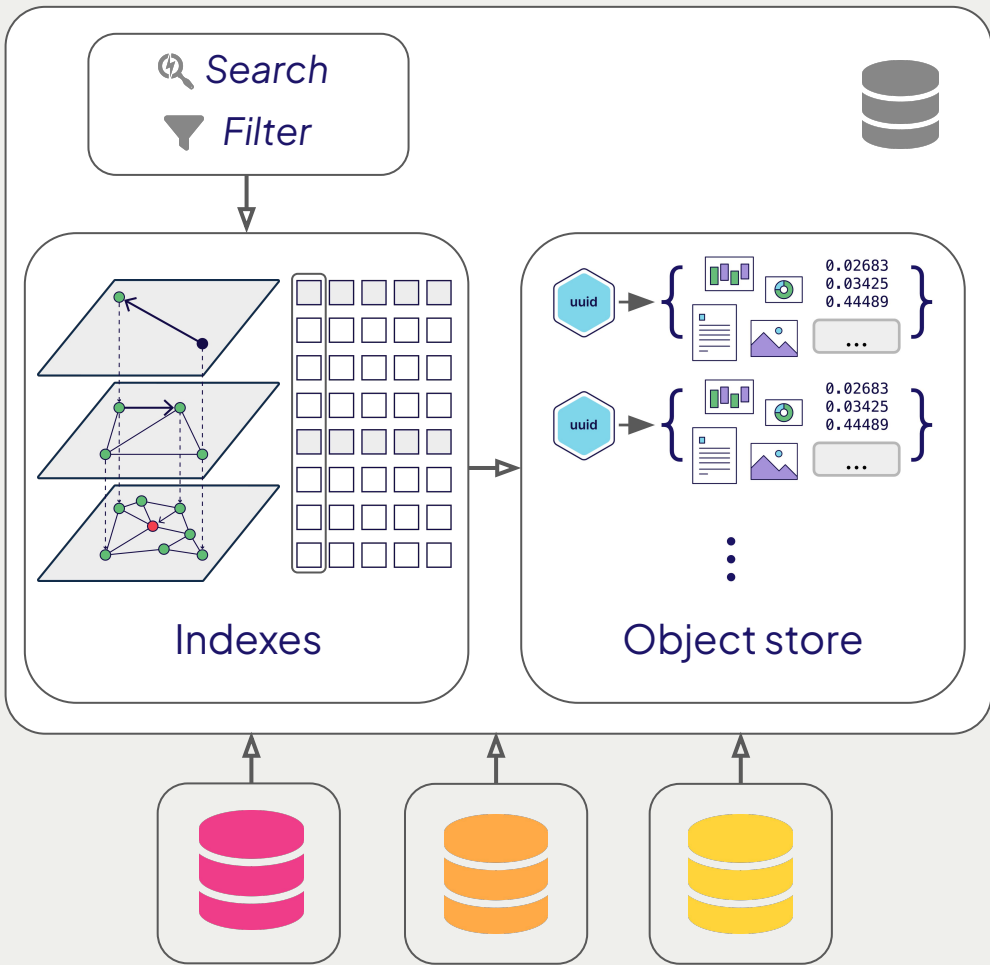
Limitation:

- Scale
 - More data
 - Multiple datasets



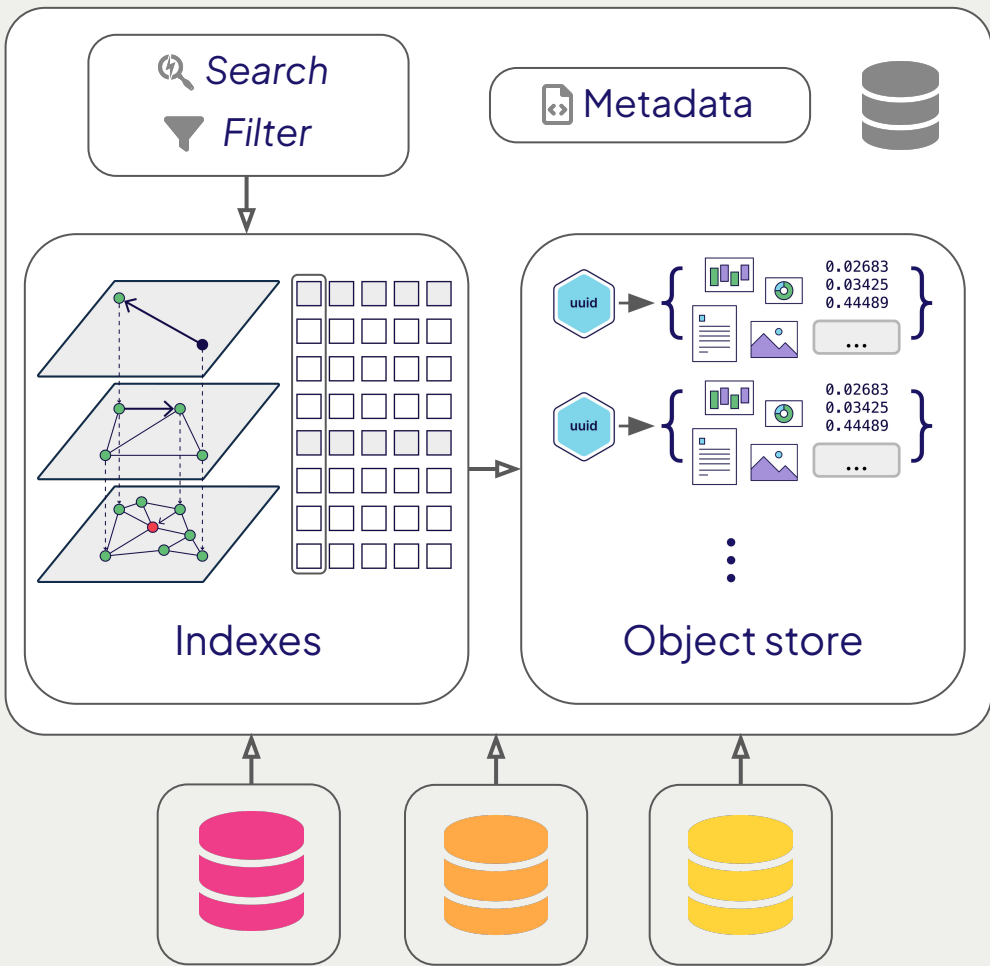
Mini-database:

- What if they could be **multiplied**?



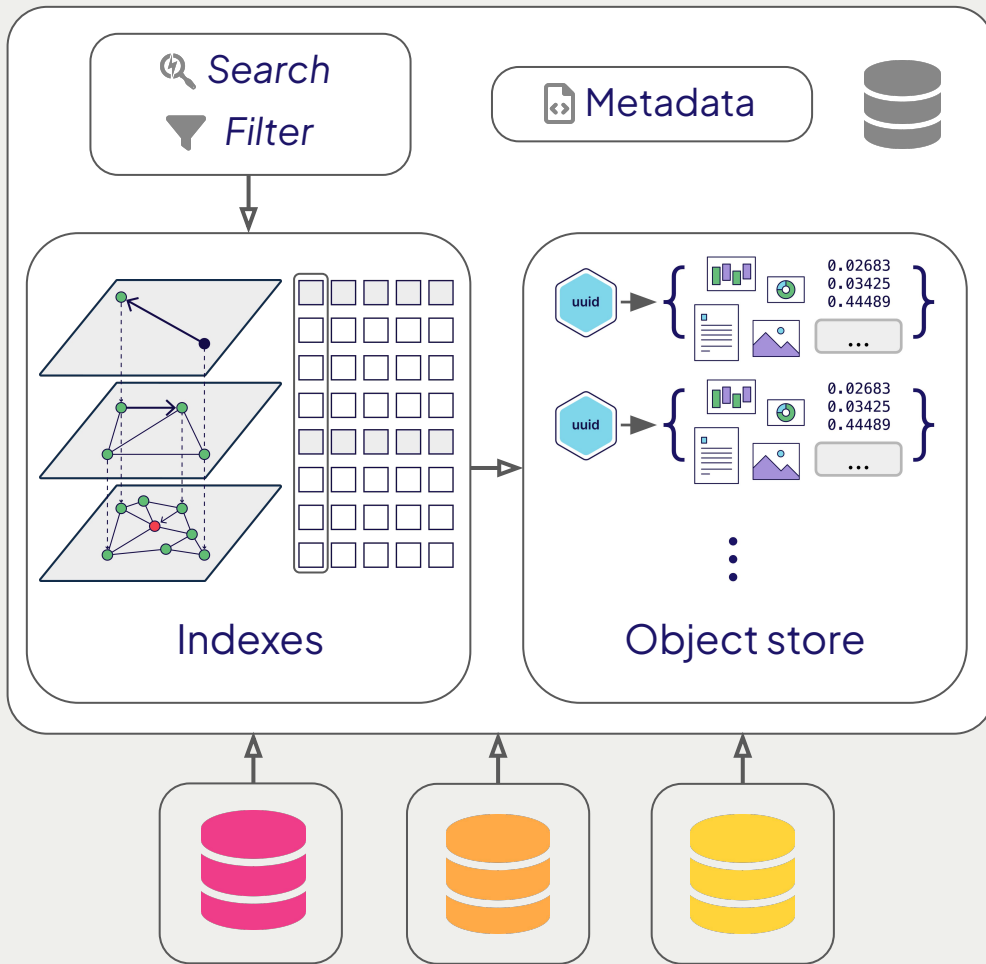
Mini-database:

- What if they could be **multiplied?**
- Could be added across multiple devices



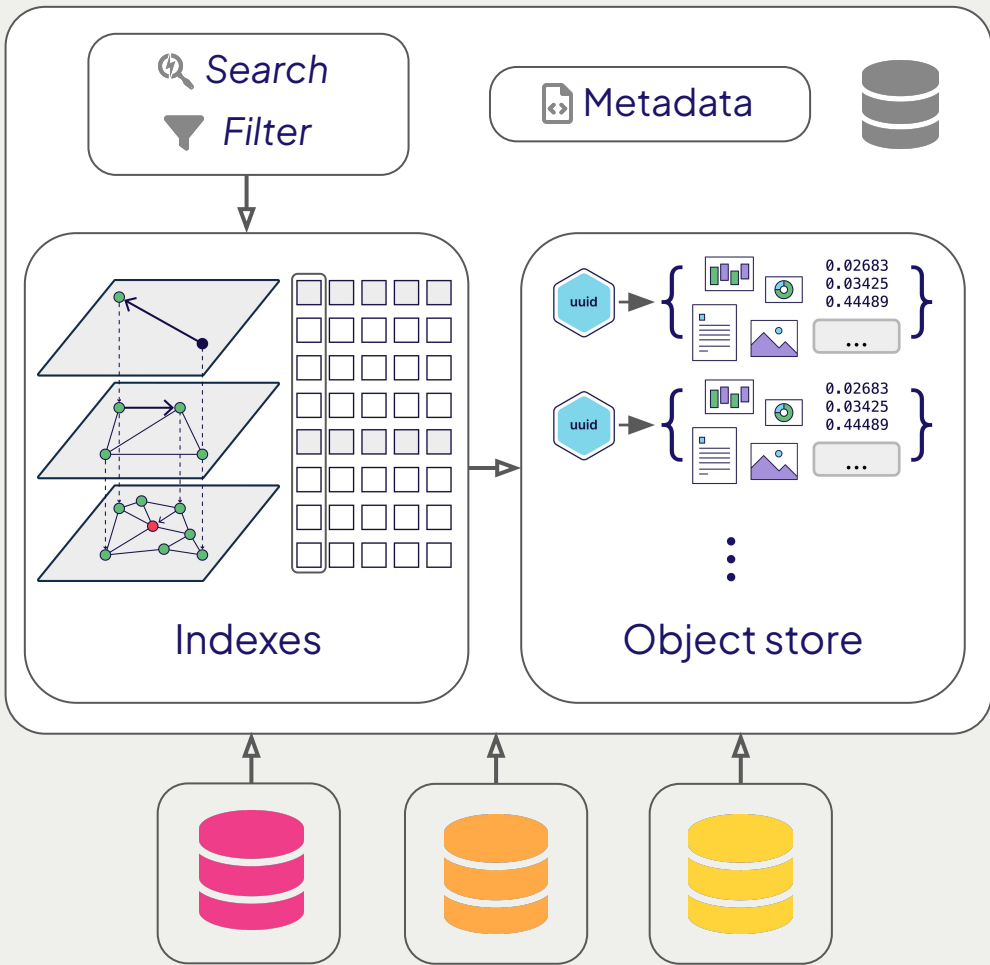
Mini-database:

- What if they could be **multiplied?**
- Could be added across multiple devices
- Called "shards" in distributed databases



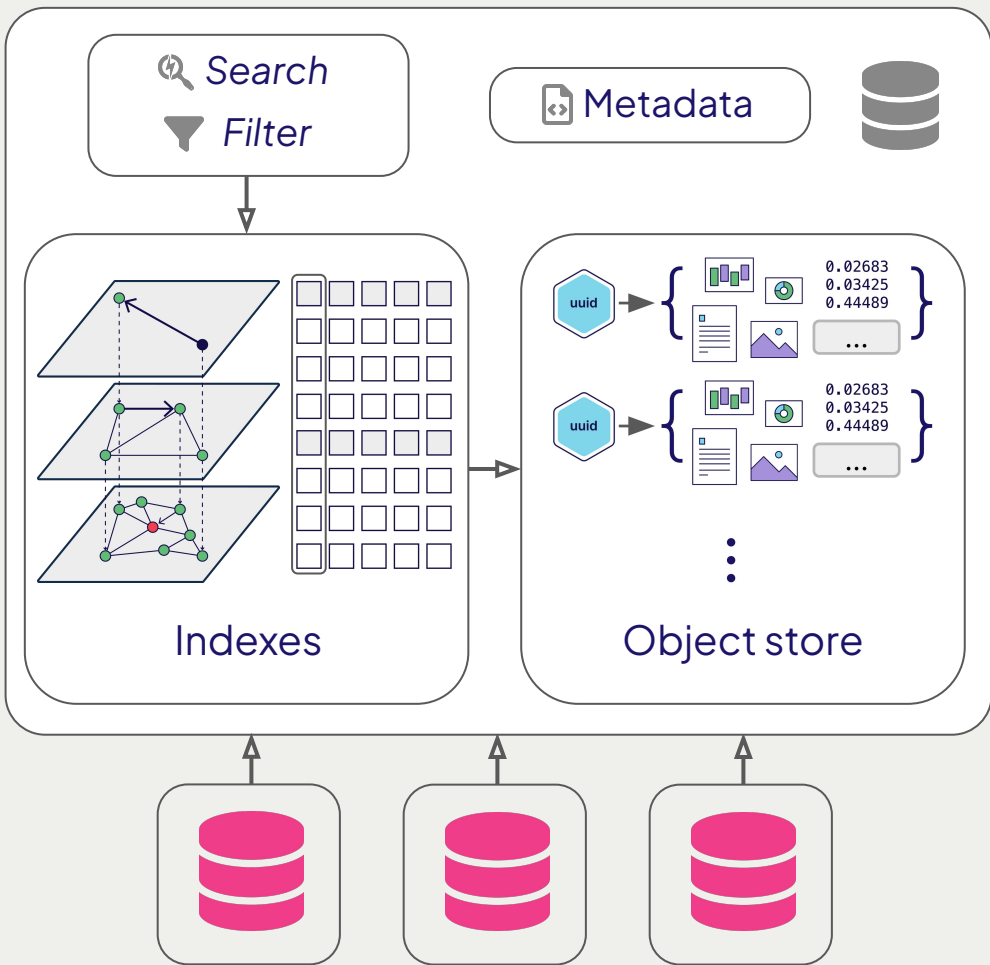
Mini-database:

- What if they could be **multiplied**?
- Could be added across multiple devices
- Called "shards" in distributed databases
- Solves a lot of issues!



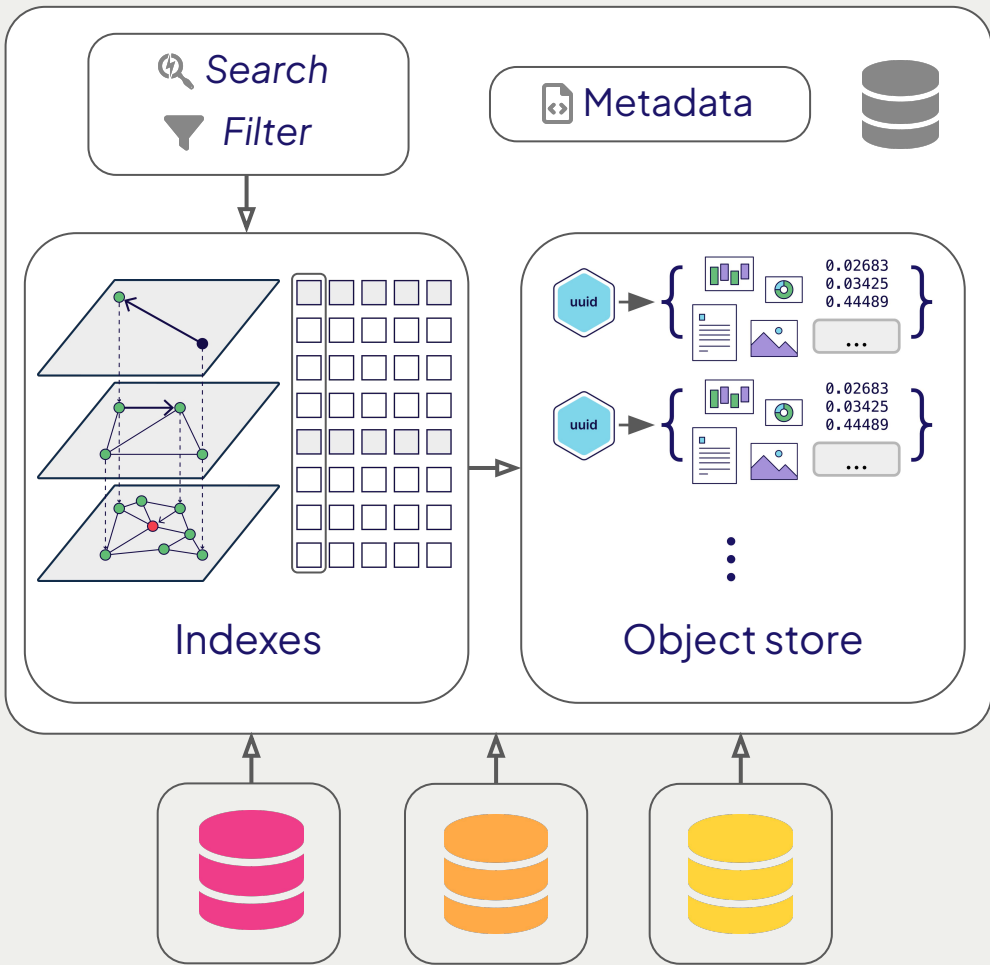
Each shard can:

- Be part of a collection
 - Horizontal scaling
- Each search goes to every shard
- Results combined



Each shard can:

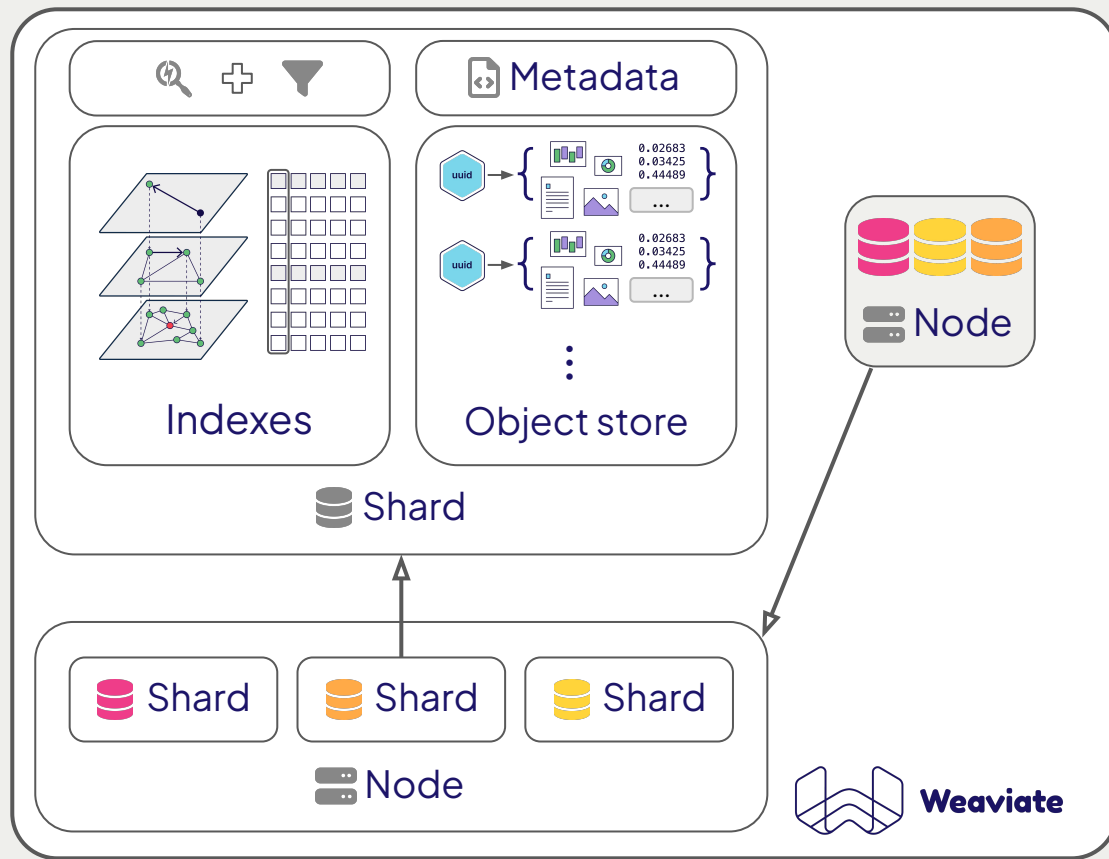
- Have "clones"
 - Replication
- Coordinate database schema
- Keep objects consistent



Each shard has:

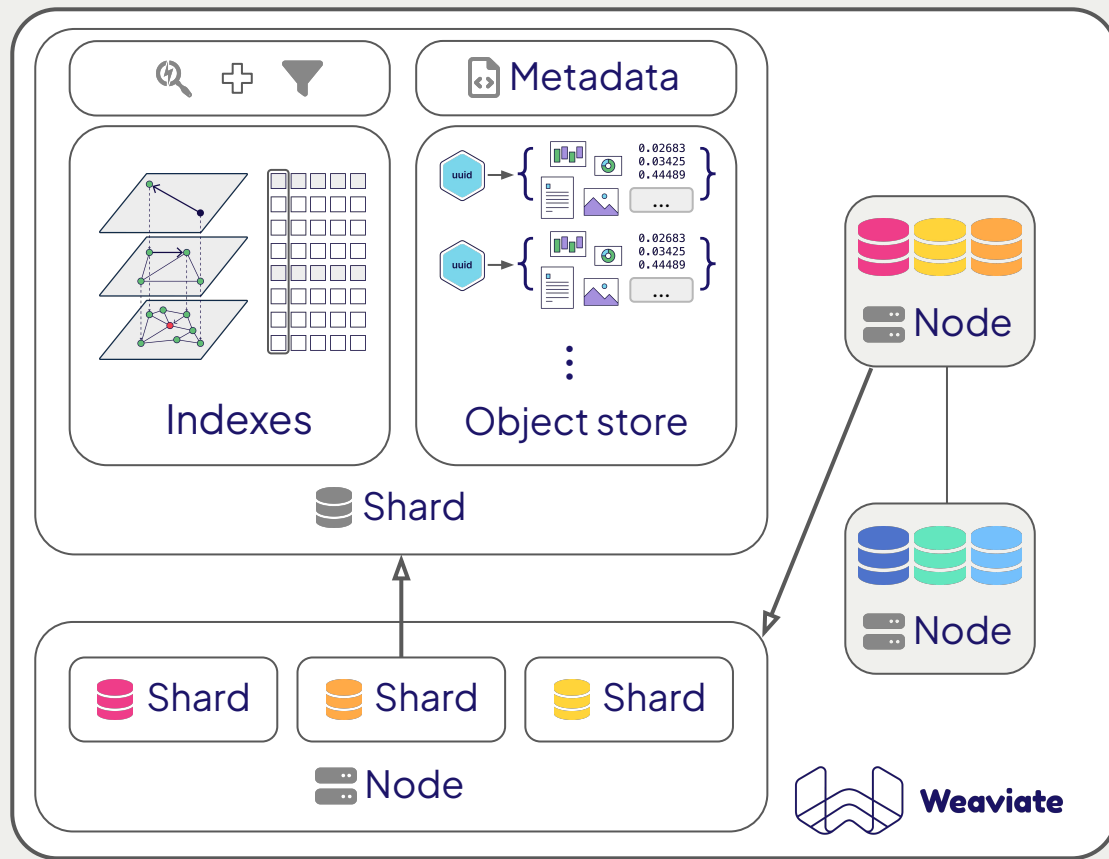
- An object store
- A vector index
- Inverted indexes
 - Searchable (keyword)
 - Filterable

How do we get BIGGER?



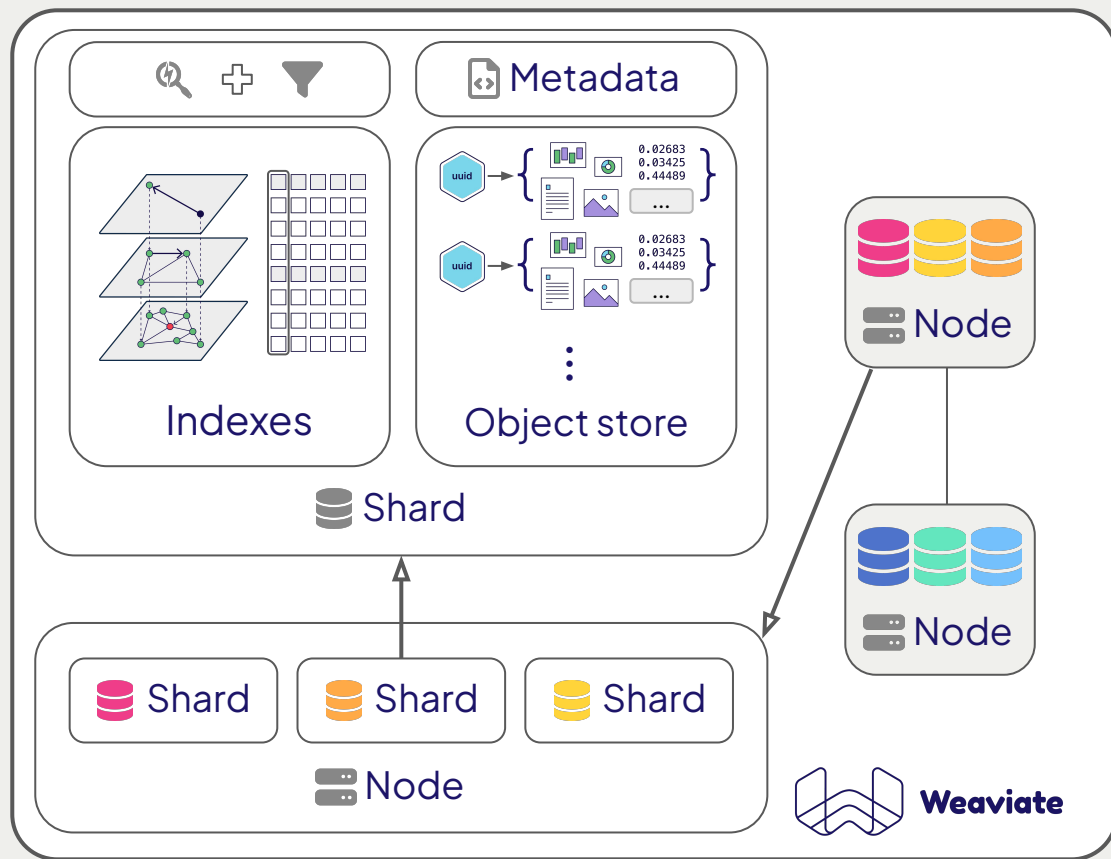
Each node:

- Can have multiple shards



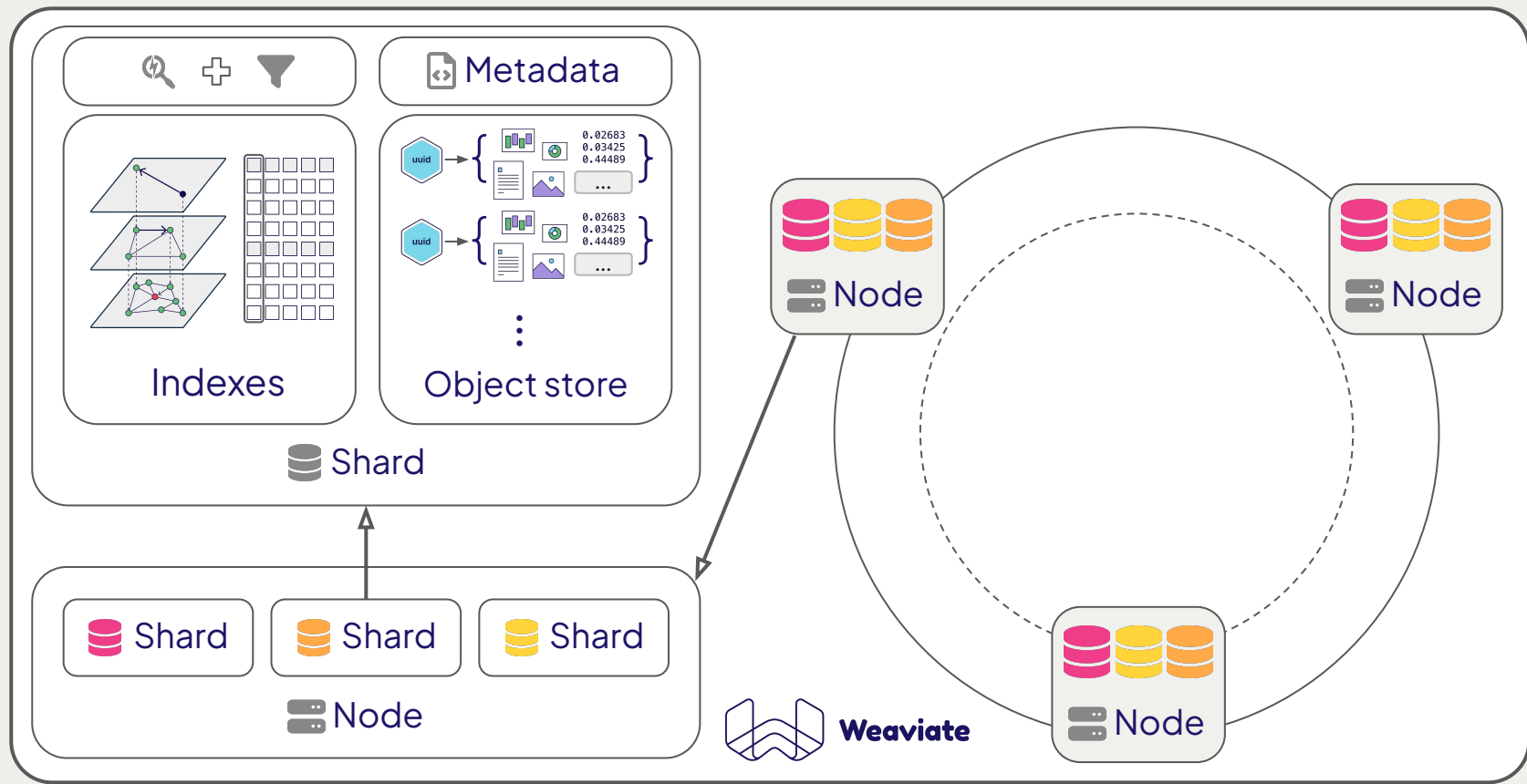
Each node:

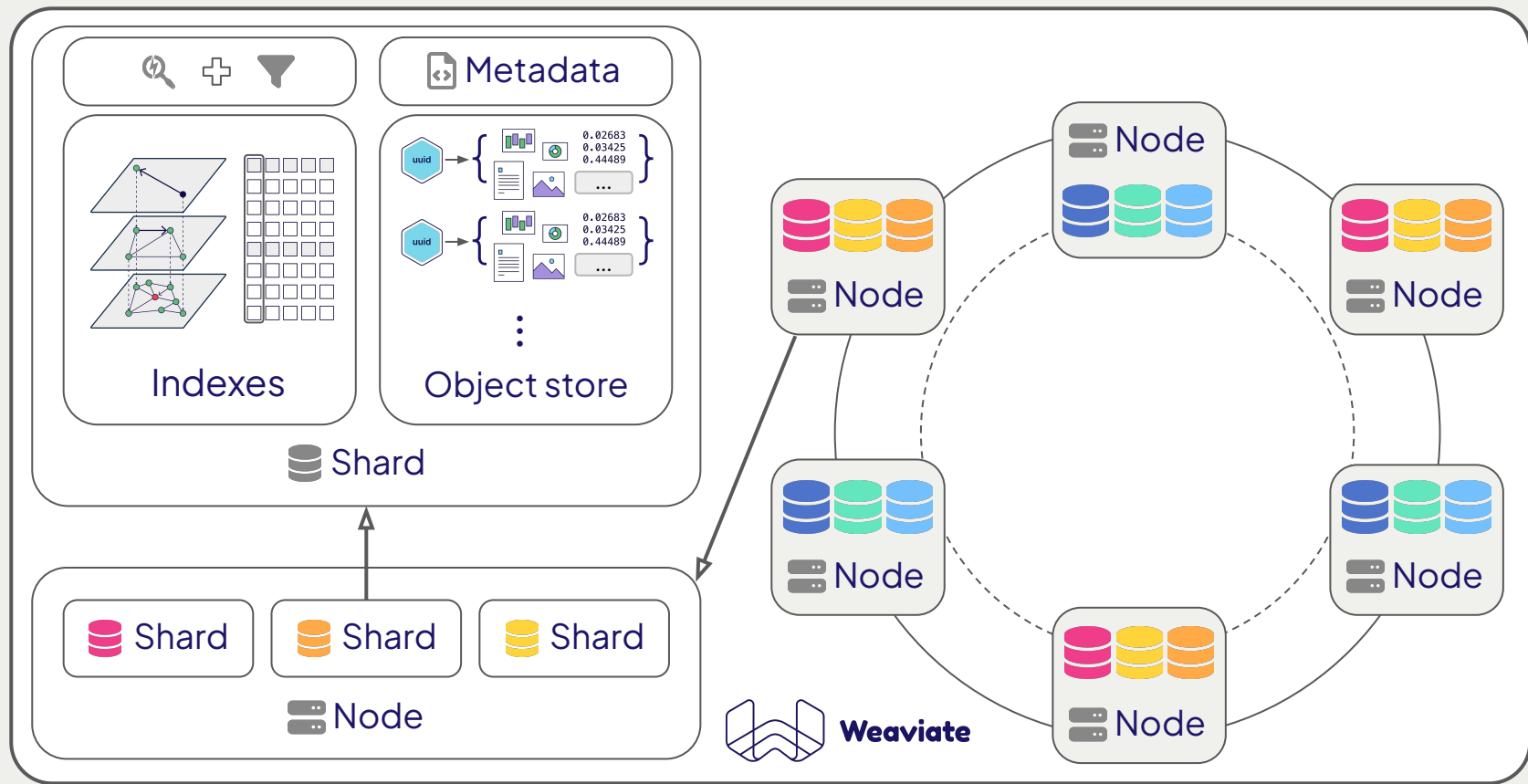
- Can have multiple shards
- Can also talk to other nodes

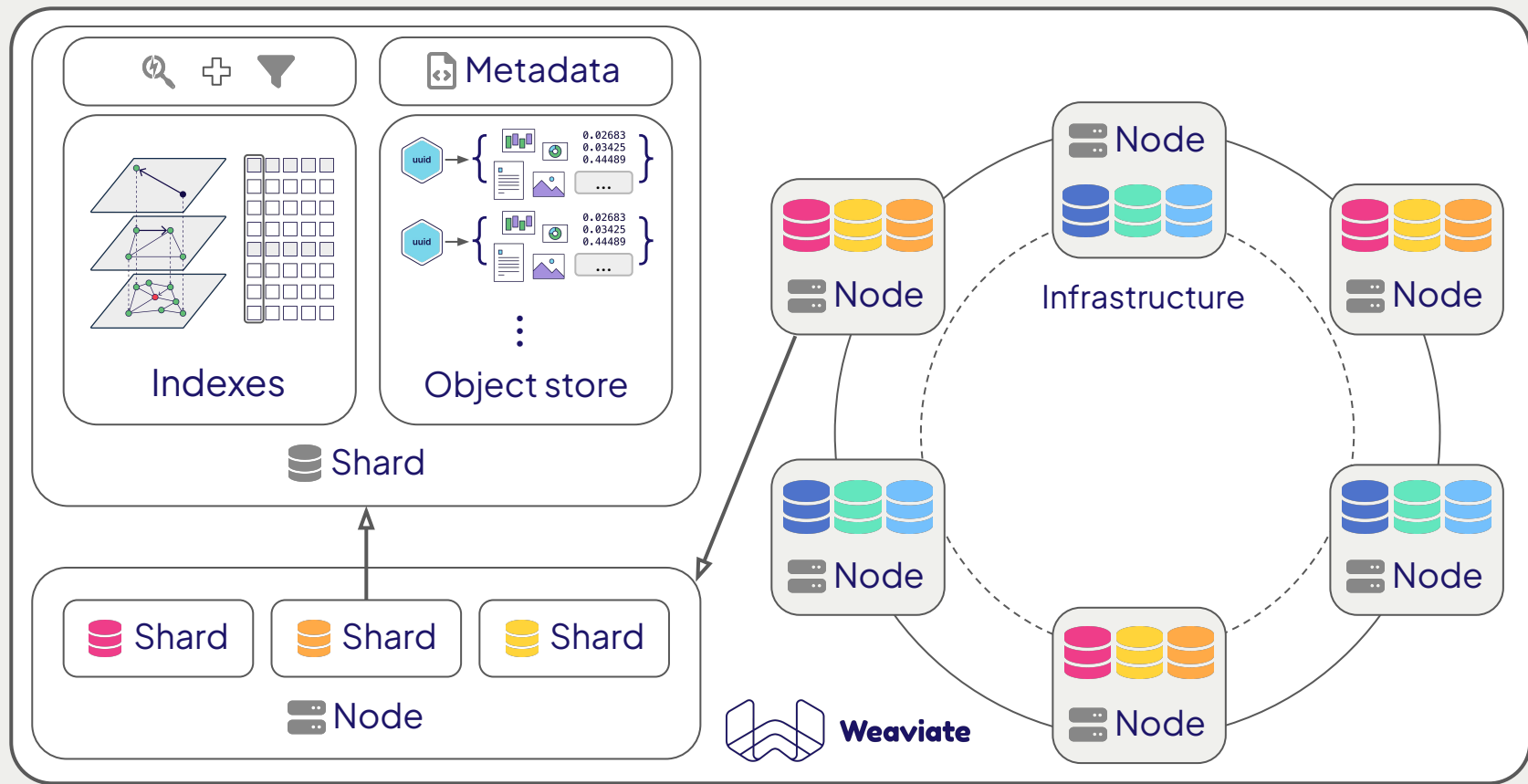


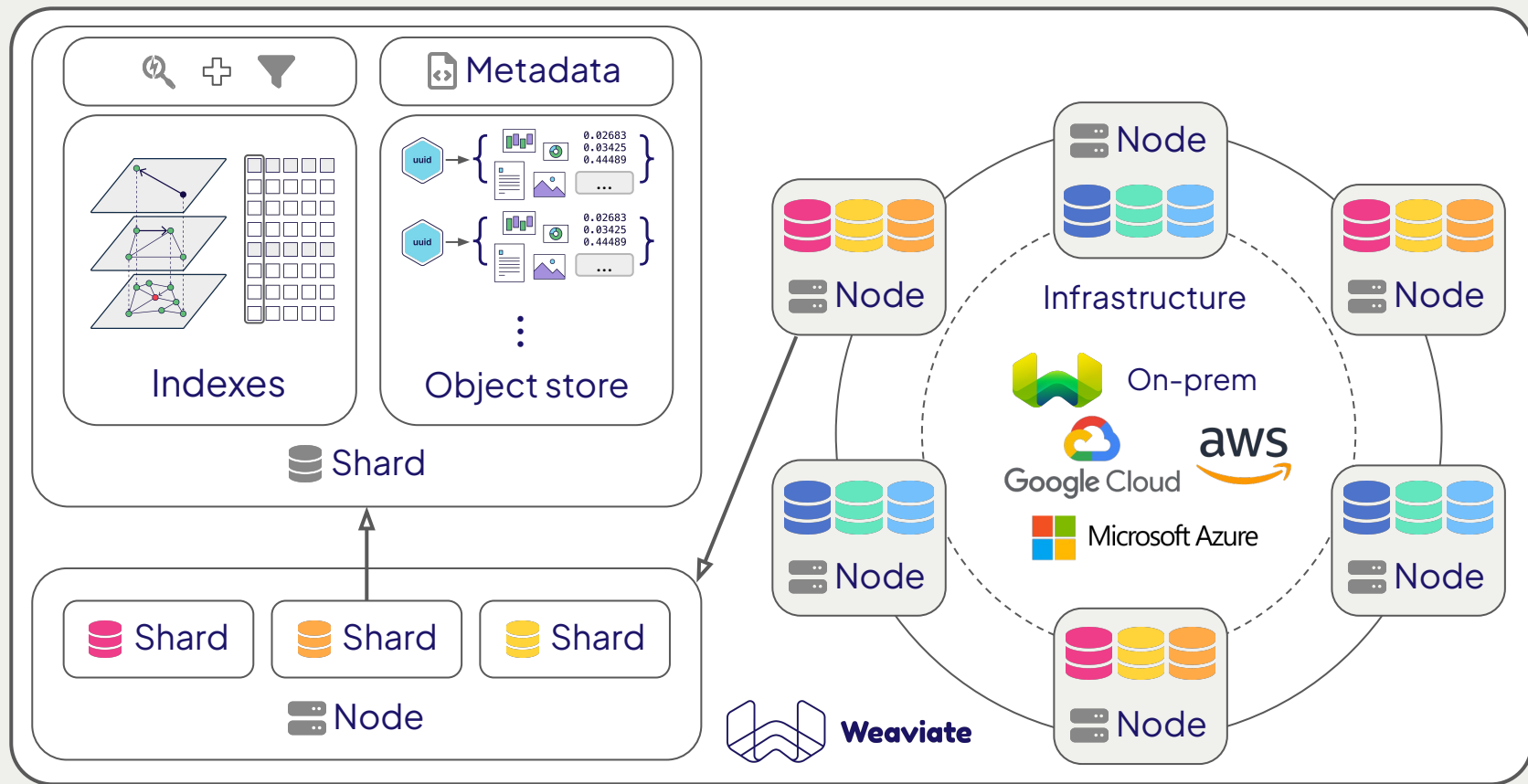
User perspective:

- Resource planning
 - How many replicas?
 - How many objects?
 - What/any compression?





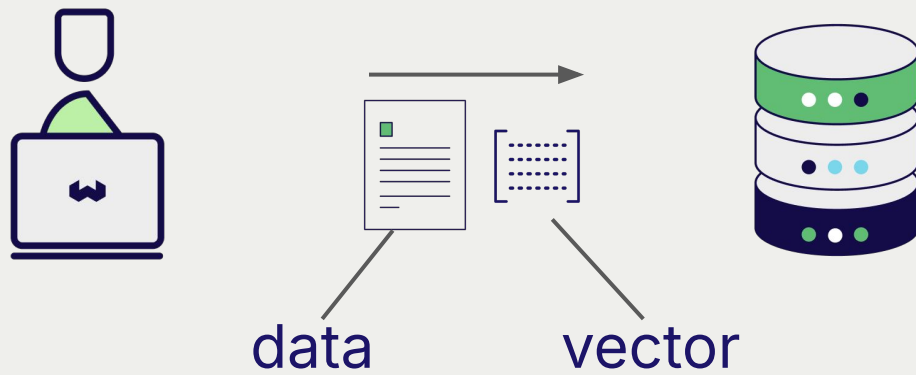




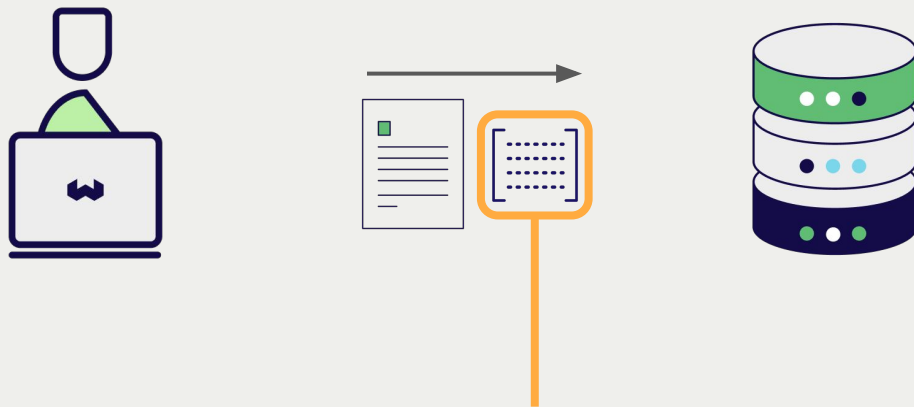


But wait... there's more!

Ingest data

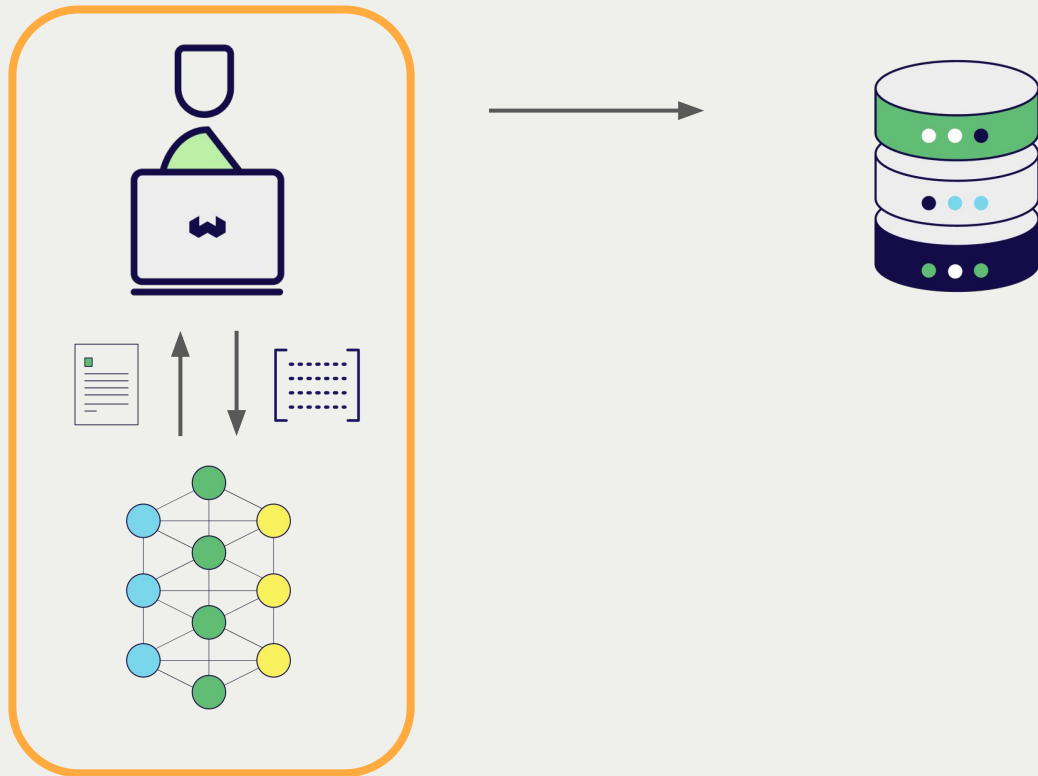


Ingest data

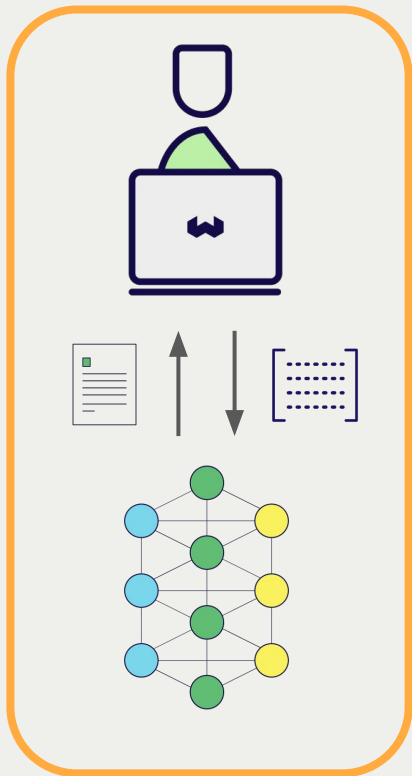


Where do they come from??

Obtain Embeddings



Obtain Embeddings



```
import cohere

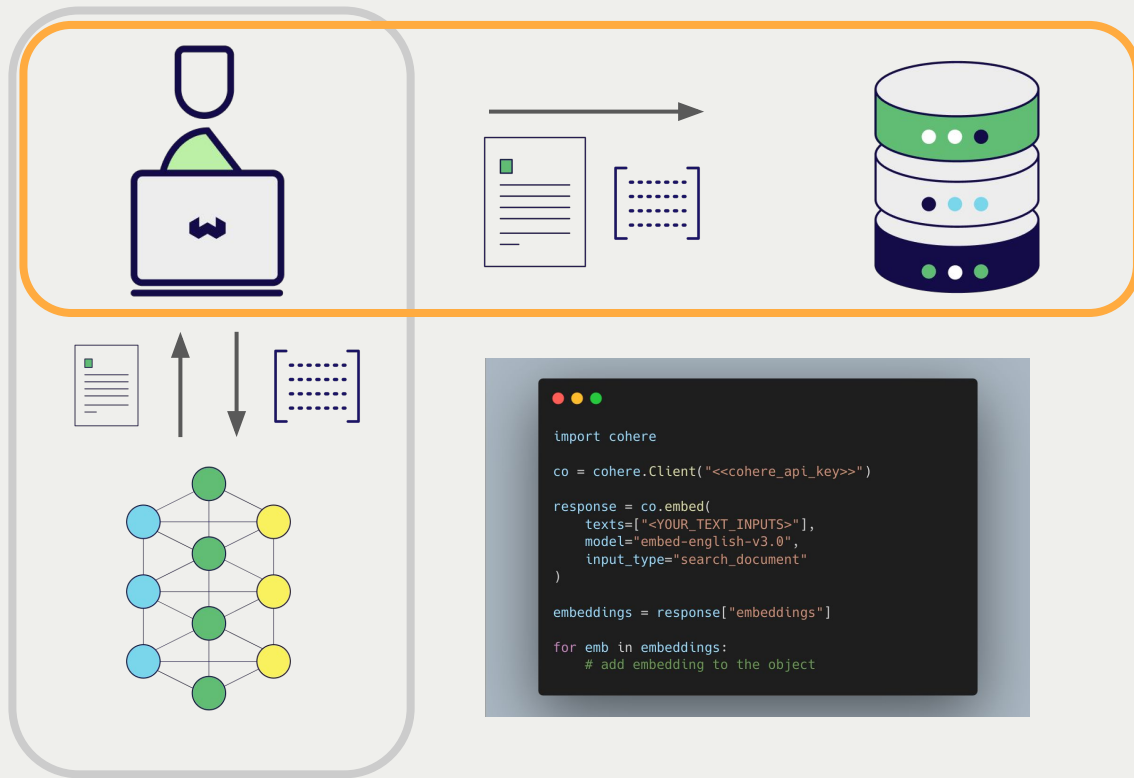
co = cohere.Client("<<cohere_api_key>>")

response = co.embed(
    texts=["<YOUR_TEXT_INPUTS>"],
    model="embed-english-v3.0",
    input_type="search_document"
)

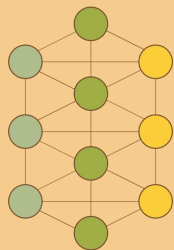
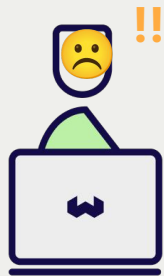
embeddings = response["embeddings"]

for emb in embeddings:
    # add embedding to the object
```

Obtain Embeddings



Overhead



```
import cohere

co = cohere.Client("<<cohere_api_key>>")

response = co.embed(
    texts=["<YOUR_TEXT_INPUTS>"],
    model="embed-english-v3.0",
    input_type="search_document"
)

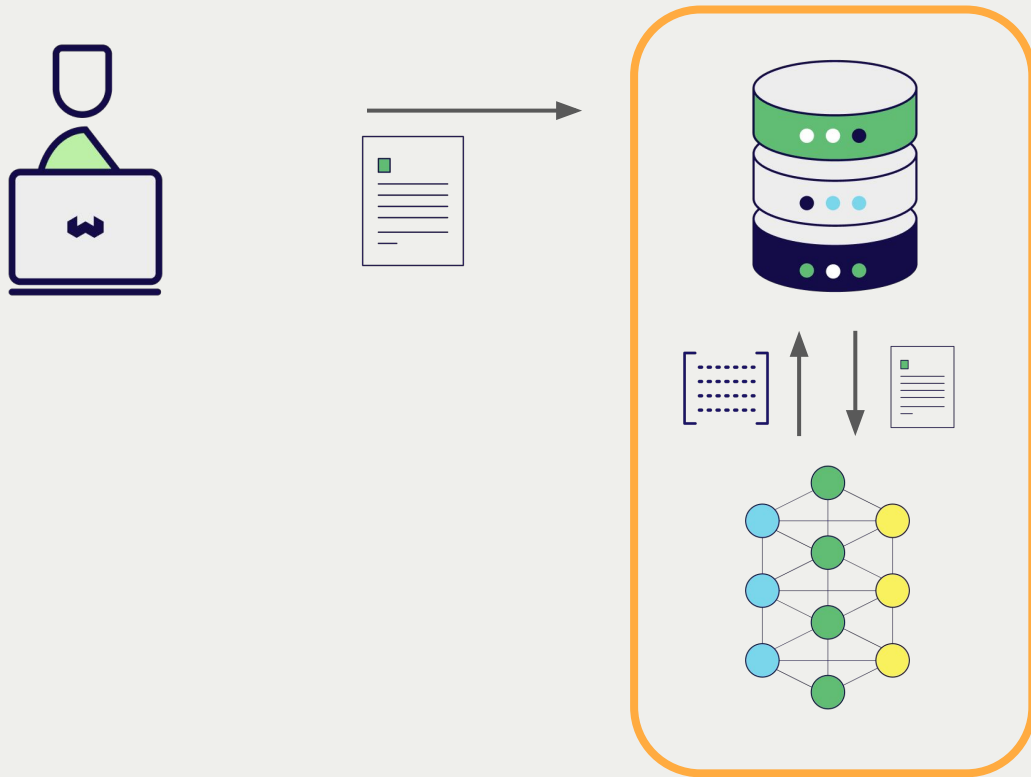
embeddings = response["embeddings"]

for emb in embeddings:
    # add embedding to the object
```

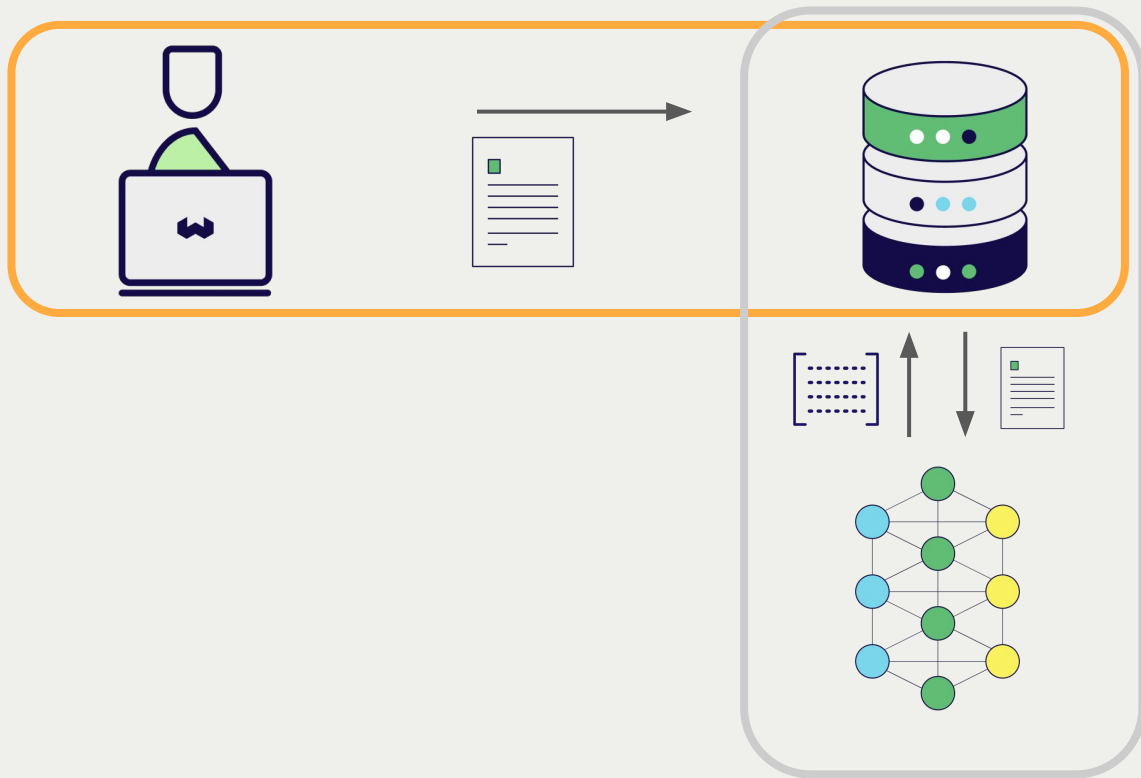
Ingest data



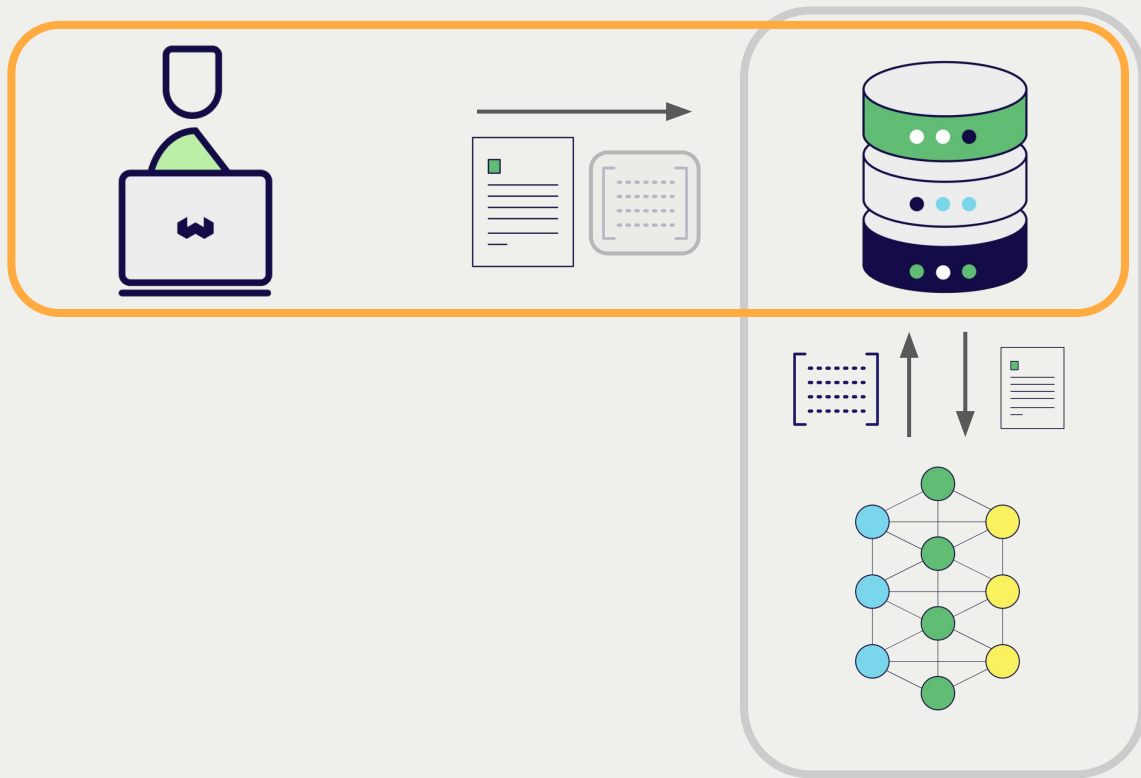
Have DB obtain embeddings



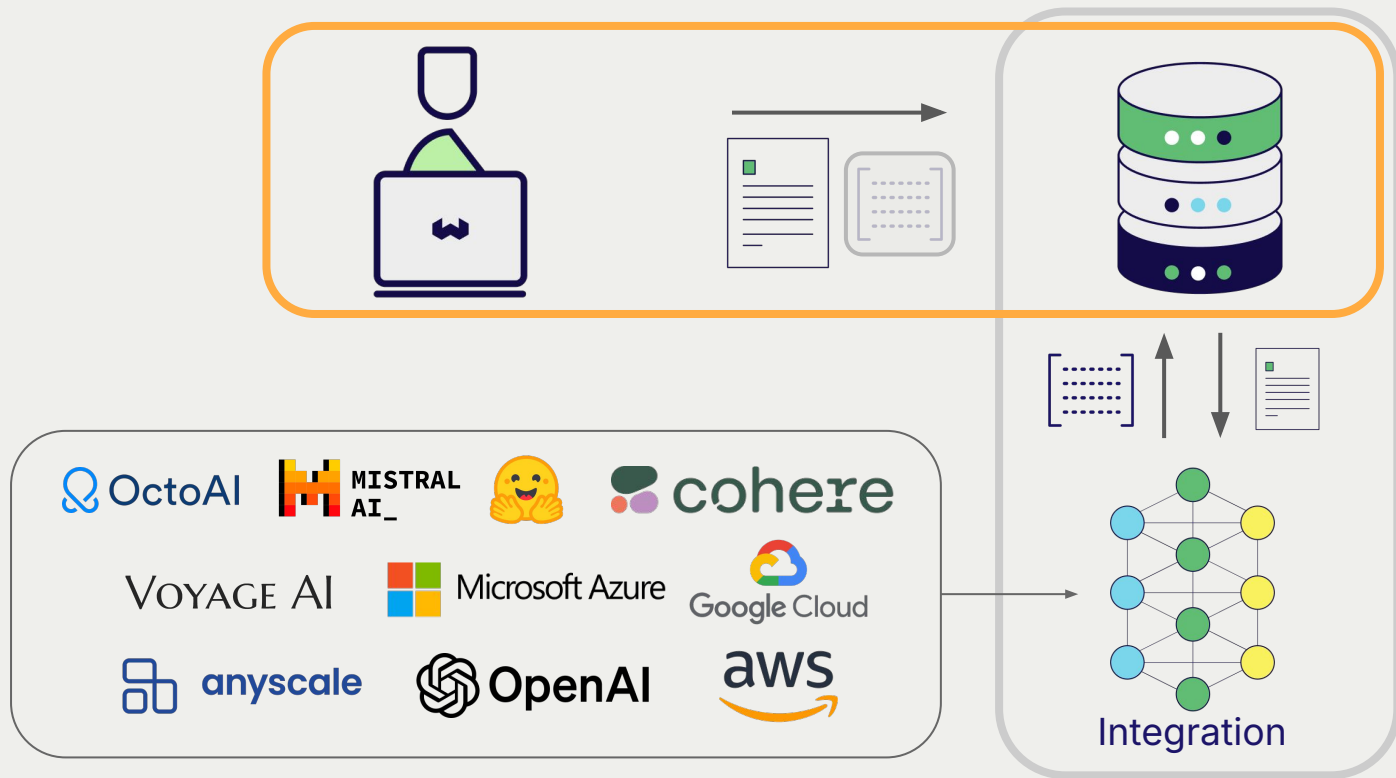
Ingest data (embeddings in background)



Ingest data (embeddings optional)

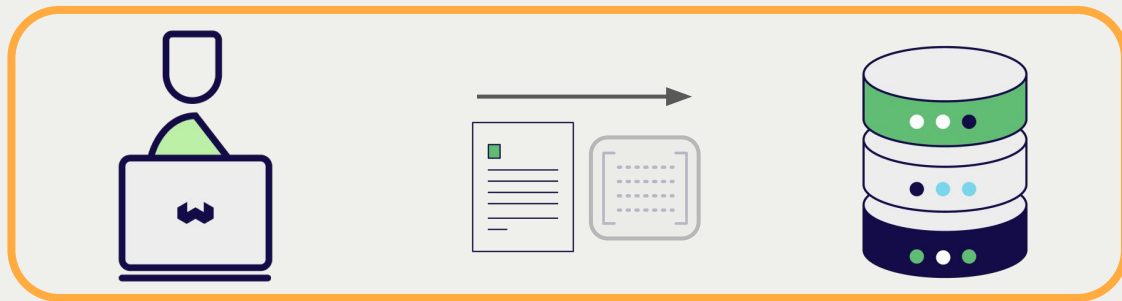


Ingest data (with provider integrations)

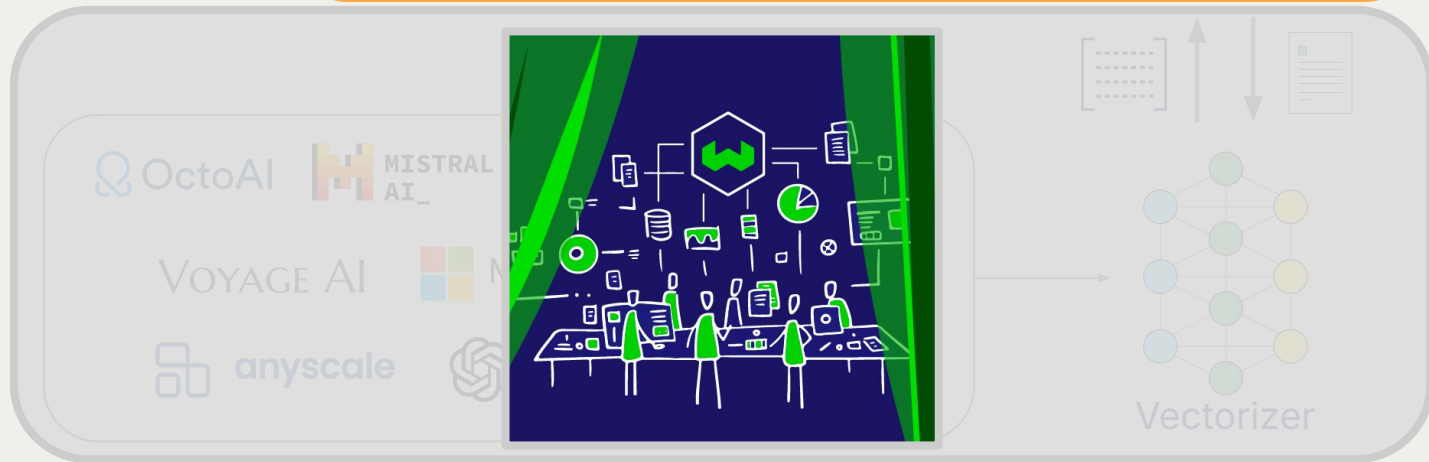




Simplified Experience

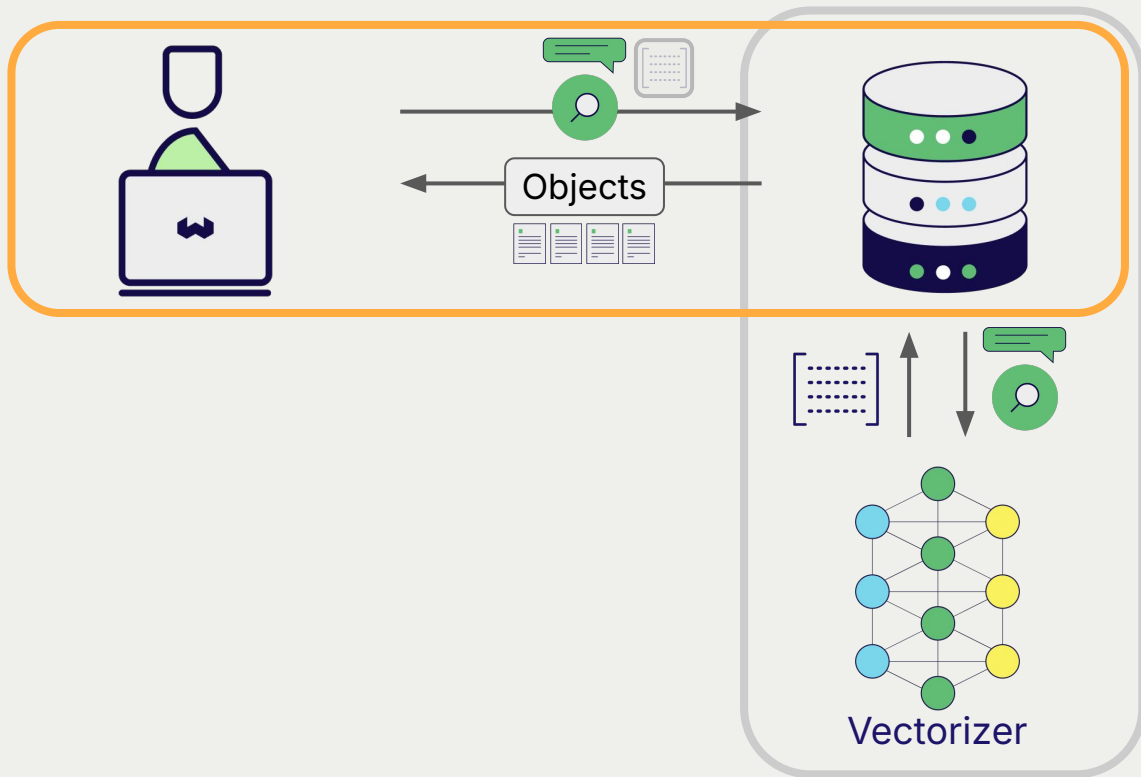


User
experience



Under the
hood

Query



Integrations



VOYAGE AI

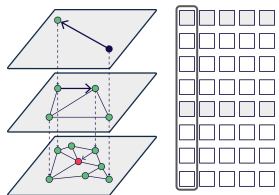


cohere

Model providers



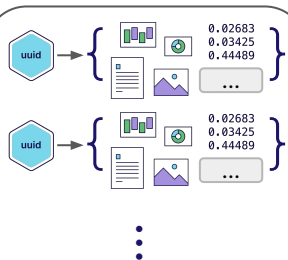
Metadata



Indexes



Shard



Shard



Shard



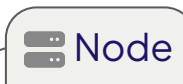
Shard



Node



Node



Node



Node



Node



Node



Node

Infrastructure



Google Cloud



Microsoft Azure



Weaviate

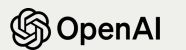


What have we built? (Full recap)

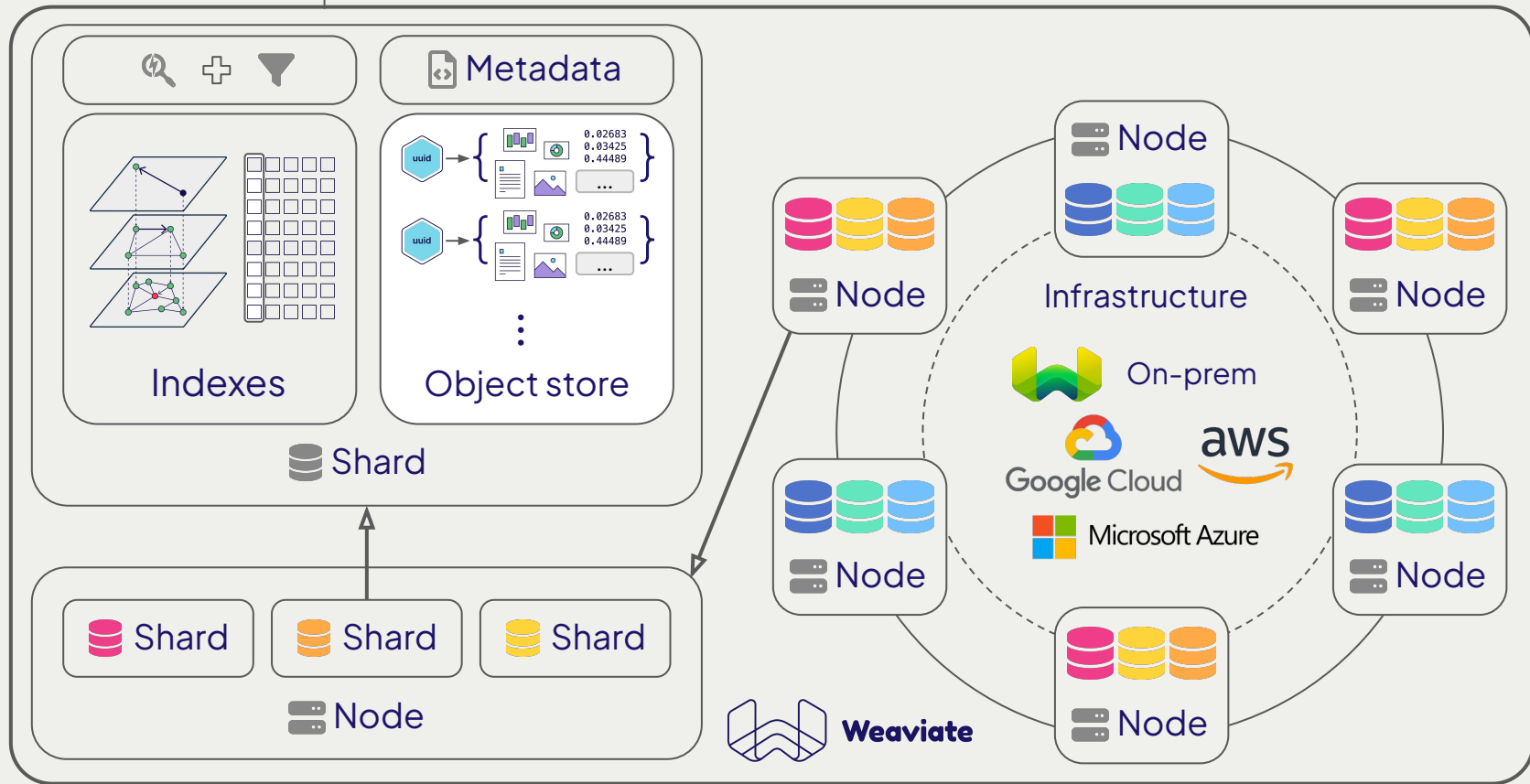
Integrations



VOYAGE AI



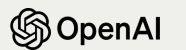
Model providers



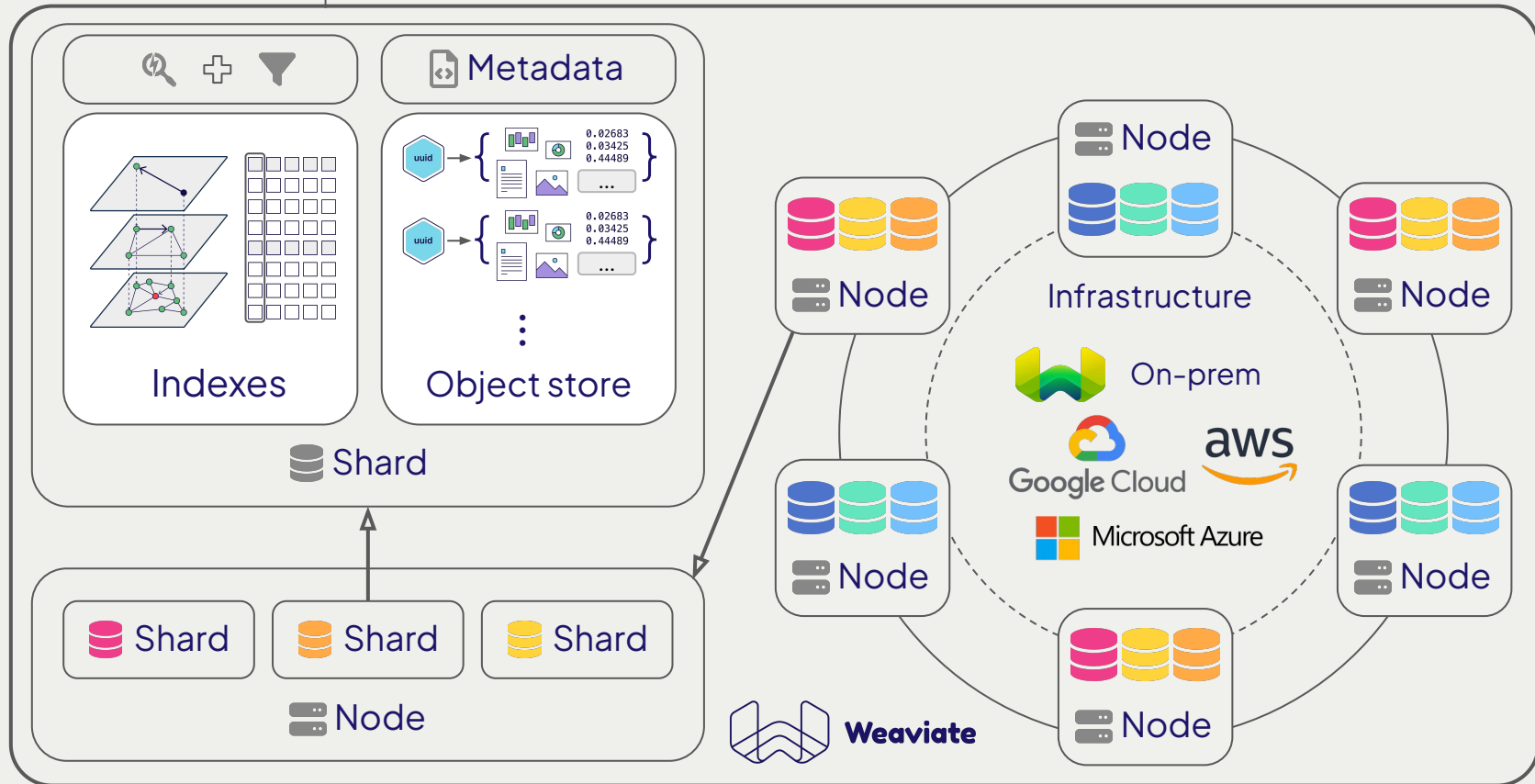
Integrations



VOYAGE AI



Model providers



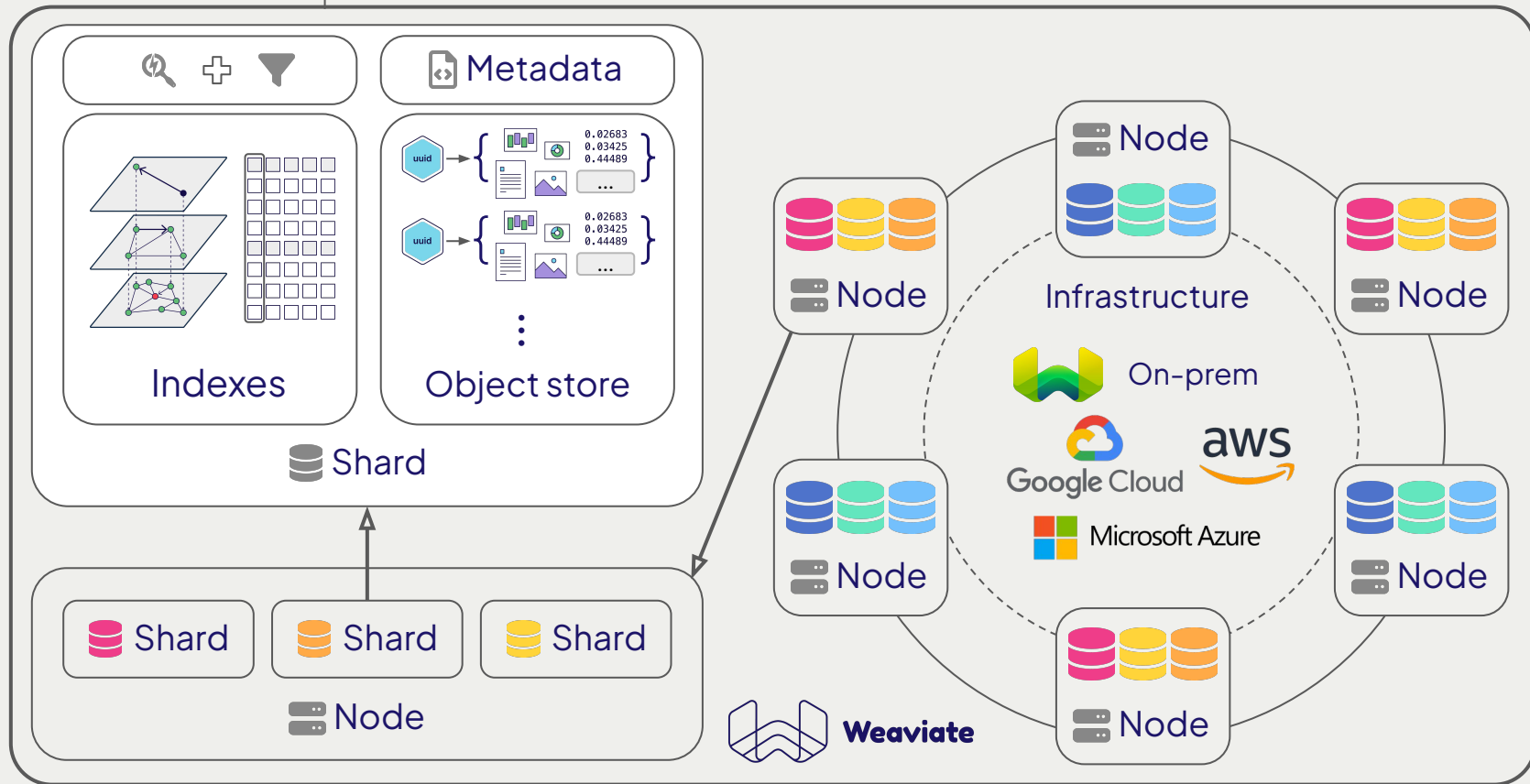
Integrations



VOYAGE AI



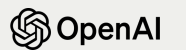
Model providers



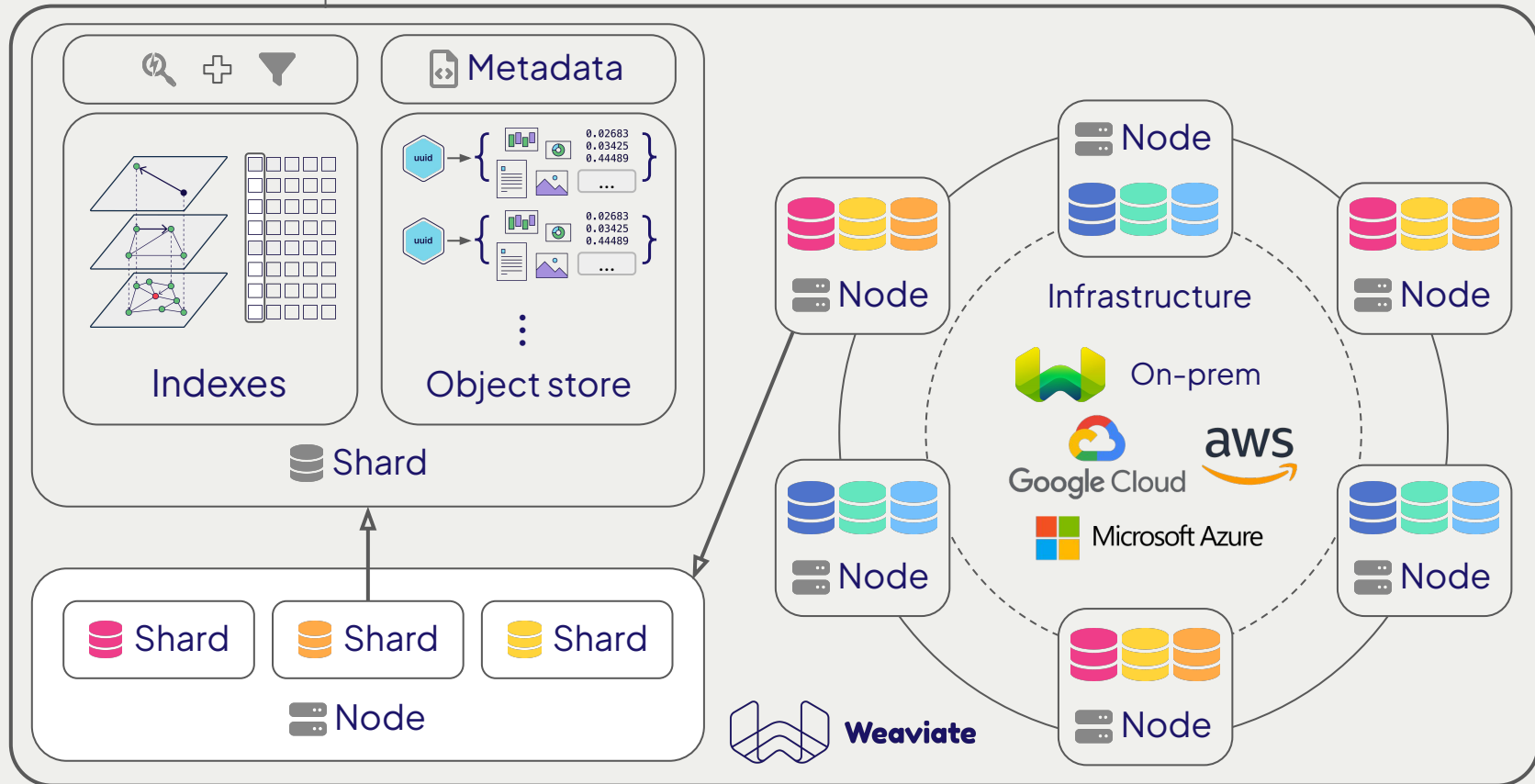
Integrations



VOYAGE AI



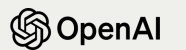
Model providers



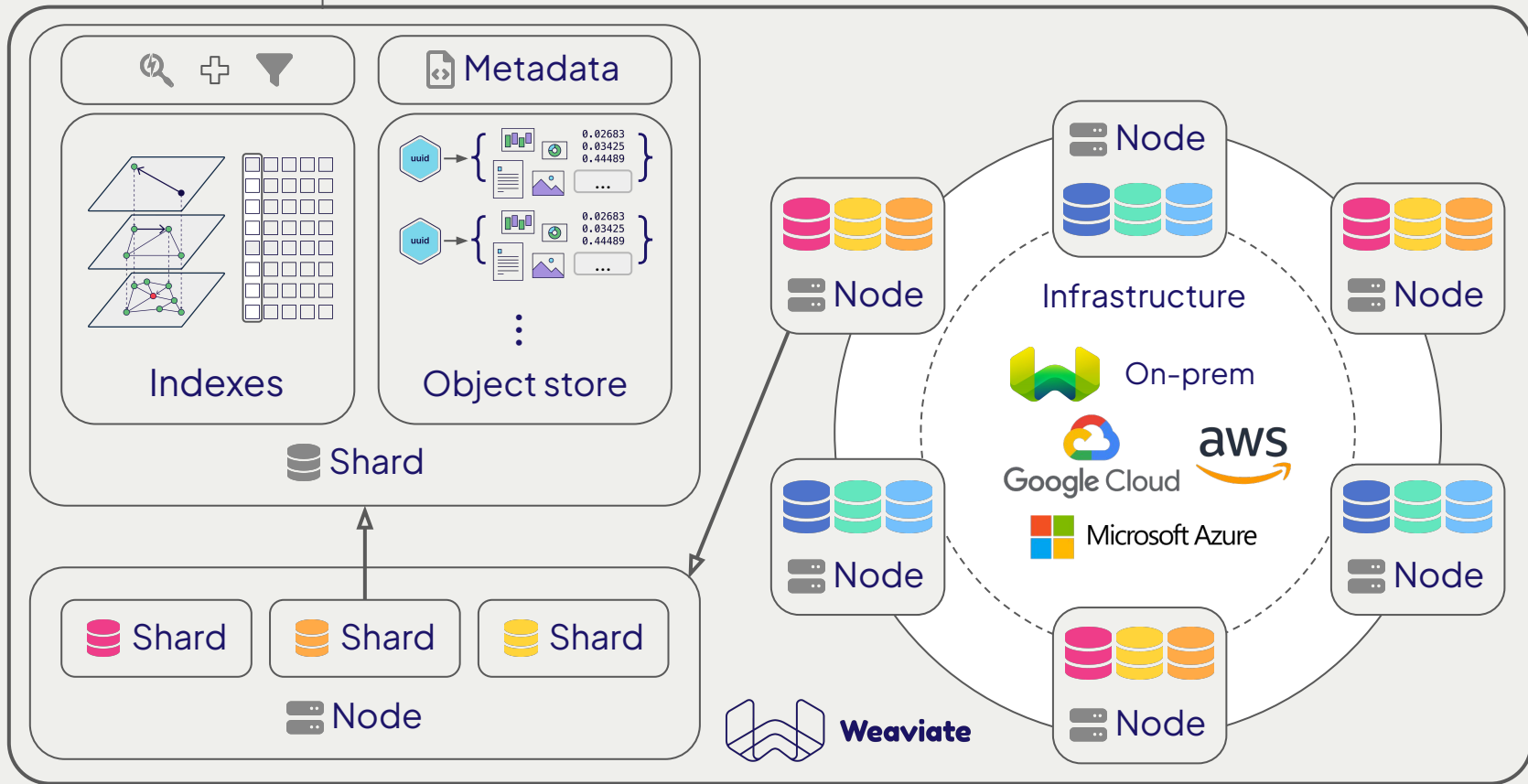
Integrations



VOYAGE AI



Model providers



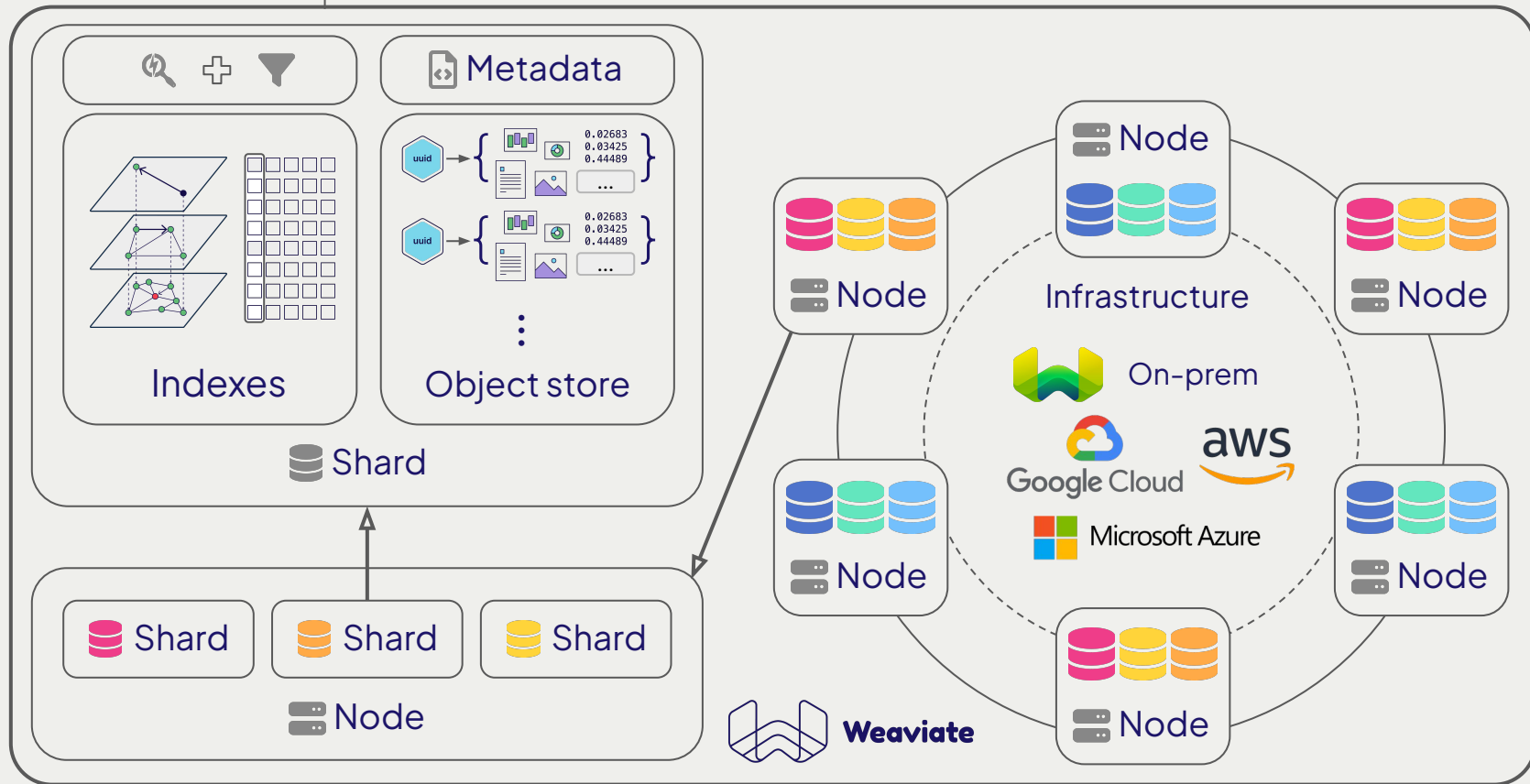
Integrations



VOYAGE AI



Model providers



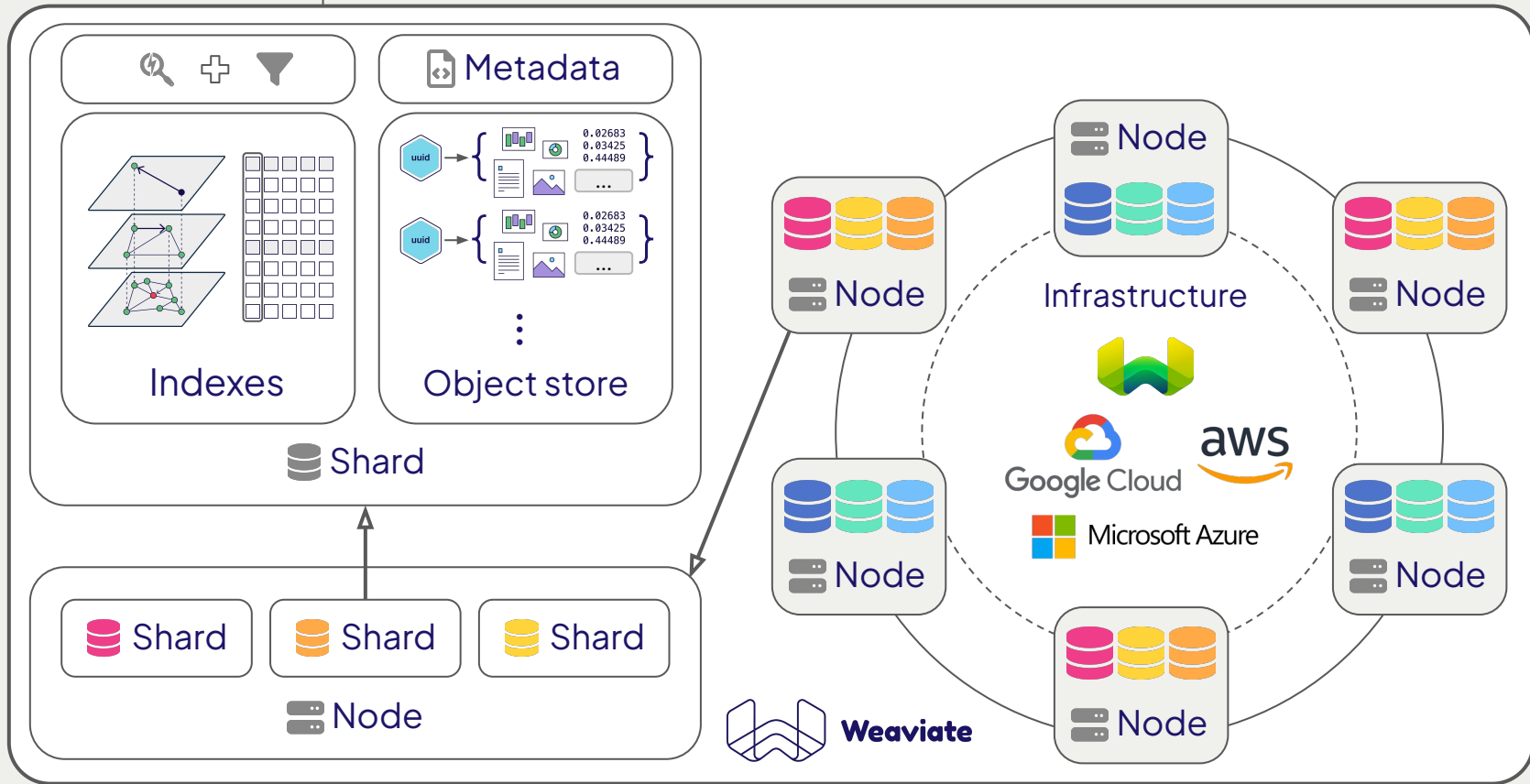
Integrations



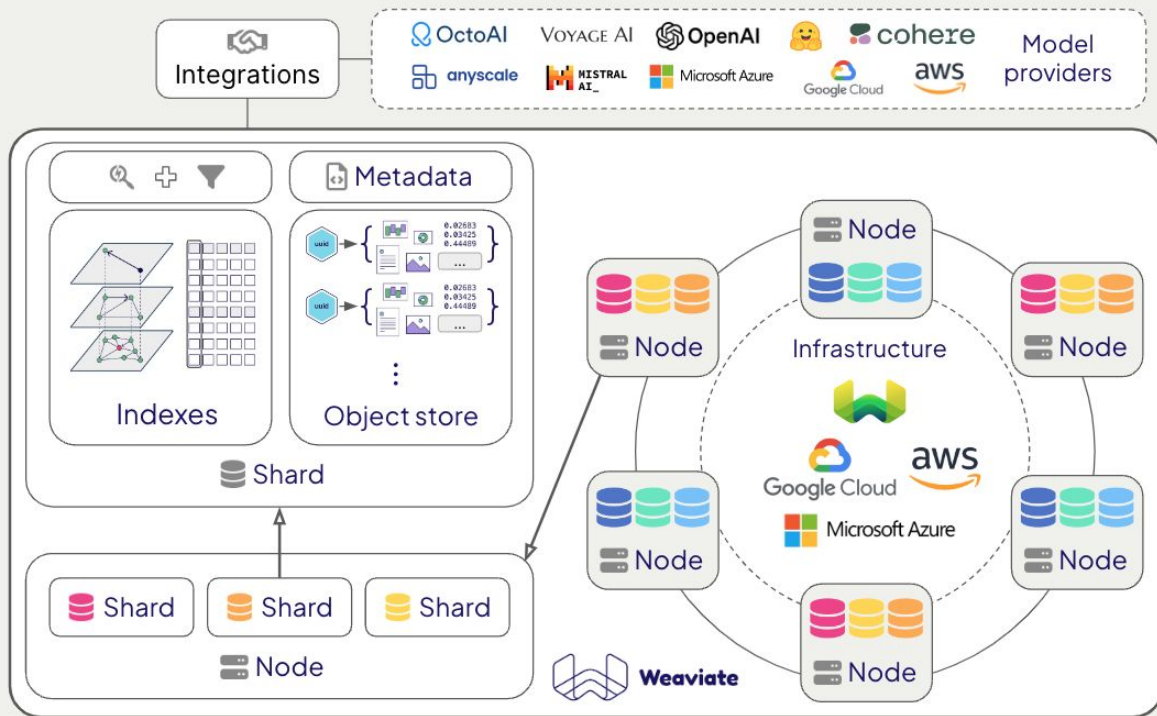
VOYAGE AI



Model providers



Weaviate



Try 



<https://hubs.ly/Q02tMNDH0>